# Dream Chalet Developer Manual

A complete, beginner-friendly guide to every library and technique used to build a premium rental property website — from GSAP animations to Three.js 3D, smooth scroll, interactive maps, multilingual support, and more.

13 Chapters **Stack:** GSAP · Three.js · Lenis · MapLibre · i18next · ical.js **Level:** Beginner → Intermediate

---

FOUNDATION 1

## What is JavaScript?

You already know HTML and CSS. HTML is the *skeleton* of a page — the structure. CSS is the *skin* — colors, fonts, layout. JavaScript is the *brain* — it makes things happen. It responds to clicks, fetches data from the internet, animates elements, validates forms, and controls every interactive thing on a modern website.

JavaScript runs directly in the browser. You do not need to install anything. You do not need a server. You write it, the browser executes it.

## Where Does JavaScript Live?

```
<!-- Option 1: Inline (avoid for anything longer than 1 line) -->
<button onclick="alert('Hello!')">Click me</button>

<!-- Option 2: Internal <script> tag (fine for learning, avoid in production) -->
<script>
  console.log("Hello from internal script");
</script>

<!-- Option 3: External file (always do this in real projects) -->
<script src="js/main.js"></script>
```

> **Rule:** Always put your `<script>` tag just before the closing `</body>` tag. This ensures your HTML is fully loaded before JavaScript tries to interact with it.

## The Browser Console — Your Best Friend

The console is a built-in tool in every browser where you can run JavaScript live, see errors, and print messages. Open it with **F12** (or **Cmd+Option+I** on Mac) → click the **Console** tab.

```
// Print a message to the console
console.log("Hello, world!");

// Print a number
console.log(42);

// Print multiple things at once
console.log("My name is", "Ilias", "and I am", 28, "years old");

// Print an error (shows in red)
console.error("Something went wrong!");

// Print a warning (shows in yellow)
console.warn("This might be a problem");

// Print a table (great for arrays of objects)
console.table([{ name: "Meteora", km: 22 }, { name: "Elati", km: 30 }]);
```

From now on, while learning: `console.log()` everything. It is how you see what your code is doing.

## Comments

```
// Single line comment — ignored by the browser

/*
  Multi-line comment
  Great for explaining complex sections
  or temporarily disabling a block of code
*/

let price = 150; // you can put a comment after code too
```

## Your First Real Script

```
<!DOCTYPE html>
<html>
<body>
  <h1 id="greeting">Hello</h1>
  <button id="btn">Change Greeting</button>

  <script>
    // Find the elements
    const heading = document.getElementById("greeting");
    const button  = document.getElementById("btn");

    // When button is clicked, change the heading text
    button.addEventListener("click", function() {
      heading.textContent = "Welcome to Dream Chalet!";
    });
  </script>
```

```
</body>
</html>
```

In three lines of JavaScript you just made an interactive web page. That is the essence of everything we will build.

---

FOUNDATION 2

# Variables & Data Types

A variable is a named container for storing a value. Think of it like a labelled box — you put something in it, give it a name, and refer to it by that name later.

## Declaring Variables

```
// const — value cannot be reassigned (use this by default)
const siteName = "Dream Chalet";
const nightlyRate = 130;

// let — value CAN be reassigned (use when value will change)
let currentGuests = 2;
currentGuests = 4; // OK — let allows reassignment

// var — old way, avoid it in modern JavaScript
var oldStyle = "don't use this";
```

> **Rule of thumb:** Always use `const` first. If you later need to reassign it, change it to `let`. Never use `var`.

## Data Types

```
// String — text, always in quotes (single, double, or backtick)
const name     = "Dream Chalet";
const greeting = 'Καλωσήρθατε';
const message  = `Your stay is ${name}`; // backtick = template literal

// Number — integers and decimals
const rooms    = 3;
const price    = 129.99;
const discount = -0.15;

// Boolean — true or false (no quotes)
const isAvailable = true;
const isPetFriendly = true;
const hasSmoking  = false;

// null — intentionally empty (you set it to nothing)
const promoCode = null;
```

```
// undefined — not set yet (variable declared but no value assigned)
let checkoutDate;
console.log(checkoutDate); // undefined

// Array — ordered list of values (covered in F4)
const amenities = ["WiFi", "Fireplace", "Parking", "Garden"];

// Object — key-value pairs (covered in F4)
const property = {
  name: "Dream Chalet",
  bedrooms: 3,
  rating: 9.8
};
```

## Checking the Type

```
console.log(typeof "hello");    // "string"
console.log(typeof 42);         // "number"
console.log(typeof true);       // "boolean"
console.log(typeof undefined);  // "undefined"
console.log(typeof null);       // "object" ← famous JavaScript quirk!
console.log(typeof []);         // "object"
console.log(typeof {});         // "object"
```

## String Operations

```
const first = "Dream";
const last  = "Chalet";

// Concatenation (joining strings)
const full = first + " " + last;     // "Dream Chalet"

// Template literal (better way)
const full2 = `${first} ${last}`;    // "Dream Chalet"

// String methods
const name = "dream chalet trikala";
console.log(name.toUpperCase());      // "DREAM CHALET TRIKALA"
console.log(name.length);             // 20
console.log(name.includes("trikala")); // true
console.log(name.replace("dream", "beautiful dream")); // "beautiful dream chalet trikala"
console.log(name.split(" "));          // ["dream", "chalet", "trikala"]

// Multi-line string
const description = `
  Welcome to Dream Chalet.
  Located near Meteora, Greece.
  3 bedrooms, 2 bathrooms.
`;
```

## Number Operations

```
const price    = 130;
const nights   = 5;
const discount = 0.1; // 10%

const total    = price * nights;               // 650
const after    = total - (total * discount);   // 585
const rounded  = Math.round(after);            // 585
const floored  = Math.floor(3.9);              // 3
const ceiling  = Math.ceil(3.1);               // 4
const maxVal   = Math.max(100, 200, 50);       // 200
const random   = Math.random();                // 0.0 to 1.0

// Convert string to number
const input    = "150";         // came from a form field — it's a string
const asNum    = Number(input); // 150 (number)
const asNum2   = parseInt("150px"); // 150 (ignores non-numeric suffix)

// Convert number to string
const display  = String(150);   // "150"
const display2 = (9.8).toFixed(1); // "9.8" (1 decimal place)
```

## Comparison & Logic Operators

```
// Comparison (always returns true or false)
console.log(5 === 5);   // true  — strict equal (checks value AND type)
console.log(5 == "5");  // true  — loose equal (avoids this, use ===)
console.log(5 !== 3);   // true  — strict not equal
console.log(10 > 5);    // true
console.log(10 >= 10);  // true
console.log(3 < 1);     // false

// Logical operators
console.log(true && true);   // true  — AND (both must be true)
console.log(true || false);  // true  — OR (at least one must be true)
console.log(!true);          // false — NOT (inverts the boolean)

// Practical example
const isWeekend  = true;
const hasPromo   = false;
const getDiscount = isWeekend && !hasPromo; // true
```

---

FOUNDATION 3

# Functions & Scope

A function is a reusable block of code with a name. Instead of writing the same logic 10 times, you write it once inside a function and call it 10 times. Functions are the building blocks of every JavaScript program.

## Function Declaration

```
// Declare (define) the function
function greetGuest(name) {
  console.log("Welcome, " + name + "!");
}

// Call (execute) the function
greetGuest("Maria");  // Welcome, Maria!
greetGuest("Nikos");  // Welcome, Nikos!
```

## Parameters and Return Values

```
// Parameters are inputs. Return sends a value back.
function calculateTotal(pricePerNight, nights, discountPercent) {
  const subtotal = pricePerNight * nights;
  const discount = subtotal * (discountPercent / 100);
  const total    = subtotal - discount;
  return total; // send the result back to whoever called us
}

// Use the returned value
const cost = calculateTotal(130, 5, 10);
console.log(cost); // 585

// Default parameters — value used when argument is not provided
function greet(name, lang = "el") {
  if (lang === "el") return "Καλωσήρθατε, " + name;
  return "Welcome, " + name;
}
console.log(greet("Maria"));        // Καλωσήρθατε, Maria
console.log(greet("John", "en"));   // Welcome, John
```

## Function Expressions

```
// Store a function in a variable
const calculateTotal = function(price, nights) {
  return price * nights;
};

console.log(calculateTotal(130, 3)); // 390
```

## Arrow Functions — Modern Syntax

Arrow functions are a shorter way to write functions. You will see them everywhere in modern JavaScript — in GSAP callbacks, event listeners, array methods.

```
// Regular function
function double(n) { return n * 2; }

// Arrow function — equivalent
const double = (n) => { return n * 2; };

// Even shorter: if body is just a return, skip the braces and "return"
```

```
const double = n => n * 2;

console.log(double(5)); // 10

// Multiple parameters
const add = (a, b) => a + b;
console.log(add(3, 4)); // 7

// No parameters
const sayHello = () => console.log("Hello!");
sayHello();

// Arrow function in a callback (very common)
setTimeout(() => {
  console.log("1 second passed!");
}, 1000);
```

## Scope — Where Variables Live

```
// Global scope — declared outside any function, accessible everywhere
const siteName = "Dream Chalet"; // global

function showSite() {
  console.log(siteName); // ✓ can access global variable
}

// Local (function) scope — declared inside a function, only exists there
function calculatePrice() {
  const discount = 0.1; // local — only exists inside this function
  return 130 * (1 - discount);
}

console.log(discount); // ✗ ReferenceError: discount is not defined

// Block scope — let and const declared inside {} only exist in that block
if (true) {
  let blockVar = "I only exist here";
  const alsoBlock = "me too";
}
console.log(blockVar); // ✗ ReferenceError

// Closures — a function that remembers variables from its outer scope
function makeCounter() {
  let count = 0;           // this variable is "closed over"
  return function() {
    count++;
    return count;
  };
}
const counter = makeCounter();
console.log(counter()); // 1
console.log(counter()); // 2
console.log(counter()); // 3
```

> **Common mistake:** Declaring the same variable twice with `const` or using a variable before declaring it. Always declare

> variables at the top of their scope.

## Immediately Invoked Function Expression (IIFE)

A pattern you will see in library code — a function that runs immediately as soon as it is defined. Keeps variables private and avoids polluting the global scope.

```
// Regular function — must be called separately
function init() { console.log("init!"); }
init();

// IIFE — declares AND calls in one go
(function() {
  console.log("I run immediately!");
  const privateVar = "no one outside can see me";
})();

// Arrow function IIFE
(() => {
  console.log("Also immediate!");
})();
```

---

FOUNDATION 4

# Arrays, Objects & Loops

Arrays and objects are the two most important data structures in JavaScript. Everything you do with GSAP, MapLibre, i18next, and Three.js involves working with arrays and objects. Master these and the rest follows naturally.

## Arrays

An array is an ordered list. Items are accessed by their position (index), starting from 0.

```
// Create an array
const amenities = ["WiFi", "Fireplace", "Parking", "Garden", "Kitchen"];

// Access items by index (starts at 0)
console.log(amenities[0]); // "WiFi"
console.log(amenities[2]); // "Parking"
console.log(amenities[amenities.length - 1]); // "Kitchen" (last item)

// Modify an item
amenities[1] = "Wood Fireplace";

// Array length
console.log(amenities.length); // 5
```

## Essential Array Methods

```
const prices = [130, 150, 120, 180, 140];

// push — add to end
prices.push(160);
console.log(prices); // [..., 160]

// pop — remove from end
const last = prices.pop();
console.log(last); // 160

// includes — check if value exists
console.log(prices.includes(150)); // true

// indexOf — find the position of a value
console.log(prices.indexOf(120)); // 2

// slice — get a portion (does not modify original)
const firstThree = prices.slice(0, 3); // [130, 150, 120]

// splice — remove/insert items (DOES modify original)
prices.splice(1, 1); // remove 1 item at index 1

// join — turn array into a string
const list = ["Meteora", "Elati", "Pertouli"];
console.log(list.join(", ")); // "Meteora, Elati, Pertouli"

// reverse
console.log([1,2,3].reverse()); // [3, 2, 1]

// sort
console.log([30, 5, 100].sort((a, b) => a - b)); // [5, 30, 100]
```

## Array Iteration Methods (used constantly)

```
const locations = [
  { name: "Meteora",    km: 22 },
  { name: "Elati",      km: 30 },
  { name: "Pertouli",   km: 54 },
  { name: "Palaiokarya",km: 35 }
];

// forEach — loop over every item (no return value)
locations.forEach(loc => {
  console.log(loc.name + " is " + loc.km + " km away");
});

// map — create a NEW array by transforming each item
const names = locations.map(loc => loc.name);
console.log(names); // ["Meteora", "Elati", "Pertouli", "Palaiokarya"]

// filter — create a NEW array with only items that pass a test
const nearby = locations.filter(loc => loc.km < 40);
console.log(nearby); // Meteora, Elati, Palaiokarya

// find — return the FIRST item that passes a test
```

```
const meteora = locations.find(loc => loc.name === "Meteora");
console.log(meteora); // { name: "Meteora", km: 22 }

// some — true if AT LEAST ONE item passes
console.log(locations.some(loc => loc.km > 50)); // true (Pertouli = 54)

// every — true if ALL items pass
console.log(locations.every(loc => loc.km > 10)); // true

// reduce — condense array to a single value
const totalKm = locations.reduce((sum, loc) => sum + loc.km, 0);
console.log(totalKm); // 141
```

## Objects

An object stores related data as key-value pairs. While an array is like a numbered list, an object is like a named record.

```
// Create an object
const property = {
  name: "Dream Chalet",
  bedrooms: 3,
  bathrooms: 2,
  rating: 9.8,
  petsAllowed: true,
  address: {                    // nested object
    city: "Trikala",
    country: "Greece"
  },
  amenities: ["WiFi", "Fireplace"], // array inside object
  getDescription: function() {     // function inside object = method
    return this.name + " — " + this.bedrooms + " bedrooms";
  }
};

// Access values
console.log(property.name);          // "Dream Chalet"   (dot notation)
console.log(property["bedrooms"]);   // 3                (bracket notation)
console.log(property.address.city);  // "Trikala"        (nested access)
console.log(property.amenities[0]);  // "WiFi"
console.log(property.getDescription()); // "Dream Chalet — 3 bedrooms"

// Add a new property
property.phone = "+30 6976908838";

// Delete a property
delete property.petsAllowed;

// Check if property exists
console.log("rating" in property); // true
```

## Destructuring

A cleaner way to extract values from arrays and objects.

```
// Object destructuring
const { name, bedrooms, rating } = property;
console.log(name);     // "Dream Chalet"
console.log(bedrooms); // 3

// With rename
const { name: propertyName, rating: score } = property;
console.log(propertyName); // "Dream Chalet"

// With default value
const { minStay = 2 } = property; // 2 (property.minStay doesn't exist)

// Array destructuring
const [first, second, ...rest] = ["Meteora", "Elati", "Pertouli", "Palaiokarya"];
console.log(first);  // "Meteora"
console.log(second); // "Elati"
console.log(rest);   // ["Pertouli", "Palaiokarya"]
```

## Loops

```
// for loop — classic, when you need the index
for (let i = 0; i < 5; i++) {
  console.log("Night " + (i + 1));
}

// for...of — loop over array values (cleanest for arrays)
const amenities = ["WiFi", "Fireplace", "Parking"];
for (const item of amenities) {
  console.log(item);
}

// for...in — loop over object keys
const property = { name: "Dream Chalet", rooms: 3, rating: 9.8 };
for (const key in property) {
  console.log(key + ": " + property[key]);
}
// name: Dream Chalet
// rooms: 3
// rating: 9.8

// while — loop while condition is true
let count = 0;
while (count < 3) {
  console.log("Count: " + count);
  count++; // ALWAYS increment or you get an infinite loop!
}

// Spread operator — expand array/object
const moreAmenities = [...amenities, "Pool", "Garden"];
const updatedProperty = { ...property, rating: 9.9 }; // copy + override
```

> **Common mistake:** Using `forEach` when you need a return value. `forEach` always returns `undefined`. Use `map` when you need a new array, `filter` when you need a subset, `find` when you need one item.

---

FOUNDATION 5

# The DOM — Controlling the Page

The **DOM (Document Object Model)** is the browser's internal representation of your HTML page as a tree of objects. JavaScript uses the DOM to read and change anything on the page — text, styles, attributes, entire elements.

Think of it this way: HTML is the blueprint, the DOM is the actual built building, and JavaScript is the renovator who can change anything in the building at any time.

## Selecting Elements

```
// querySelector — returns the FIRST matching element (or null)
const title   = document.querySelector("h1");            // by tag
const nav     = document.querySelector(".nav");           // by class
const btn     = document.querySelector("#book-btn");      // by id
const heroBtn = document.querySelector(".hero .btn");     // nested selector

// querySelectorAll — returns ALL matching elements (NodeList)
const cards   = document.querySelectorAll(".card");
const links   = document.querySelectorAll("nav a");

// Loop over NodeList
cards.forEach(card => {
  card.style.opacity = "0";
});

// Convert to Array (for array methods like .map, .filter)
const cardsArray = Array.from(document.querySelectorAll(".card"));

// Older methods (still work, less flexible)
const el   = document.getElementById("hero");       // by id
const els  = document.getElementsByClassName("card"); // by class
```

## Reading and Changing Content

```
const title = document.querySelector("h1");

// Read text content (plain text, no HTML)
console.log(title.textContent); // "Dream Chalet"

// Set text content (safe — escapes HTML)
title.textContent = "Dream Chalet Trikala";
```

```
// Read/set HTML content (can include tags)
const box = document.querySelector(".info");
console.log(box.innerHTML); // "3 bedrooms"
box.innerHTML = "NEW content"; // ⚠ only use with trusted content

// Reading/setting attributes
const img = document.querySelector("img");
console.log(img.getAttribute("src"));        // "hero.jpg"
img.setAttribute("src", "new-hero.webp");
img.setAttribute("alt", "Dream Chalet exterior");

// data- attributes (used a lot with GSAP and JS)
const card = document.querySelector(".attraction-card");
console.log(card.dataset.location);          // value of data-location attr
card.dataset.active = "true";               // sets data-active="true"
```

## Changing CSS Styles

```
const nav = document.querySelector(".nav");

// Inline styles (camelCase for hyphenated properties)
nav.style.backgroundColor = "#0e0b08";
nav.style.padding = "12px 2rem";
nav.style.transition = "all 0.3s ease";

// Read a computed style (what the browser actually renders)
const color = getComputedStyle(nav).backgroundColor;

// CSS Custom Properties (variables)
document.documentElement.style.setProperty("--gold", "#e8c97e");
const gold = getComputedStyle(document.documentElement).getPropertyValue("--gold");
```

## classList — The Right Way to Toggle Styles

```
const nav = document.querySelector(".nav");

// Add a class
nav.classList.add("scrolled");

// Remove a class
nav.classList.remove("scrolled");

// Toggle — add if absent, remove if present
nav.classList.toggle("open");

// Check if class exists
if (nav.classList.contains("scrolled")) {
  console.log("Nav is scrolled");
}

// Add multiple classes
nav.classList.add("visible", "animated", "gold-border");

// This is better than nav.style.X because:
// 1. Styles live in CSS where they belong
```

```
// 2. Transitions work automatically
// 3. Easy to see what state an element is in
```

## Creating and Inserting Elements

```
// Create a new element
const card = document.createElement("div");
card.classList.add("attraction-card");
card.textContent = "Meteora";

// Insert into the DOM
const container = document.querySelector(".attractions");
container.appendChild(card);           // add as LAST child
container.prepend(card);               // add as FIRST child
container.insertBefore(card, refEl);   // insert before a reference element

// Cleaner insertAdjacentHTML
container.insertAdjacentHTML("beforeend", `

    Meteora
    22 km away

`);
```

## Removing Elements

```
const el = document.querySelector(".old-banner");

// Modern way
el.remove();

// Old way (still works)
el.parentNode.removeChild(el);
```

## DOM Traversal

```
const card = document.querySelector(".card");

// Navigate the DOM tree
card.parentElement          // the parent element
card.children               // all direct children (HTMLCollection)
card.firstElementChild      // first child element
card.lastElementChild       // last child element
card.nextElementSibling     // next sibling element
card.previousElementSibling  // previous sibling element

// Practical: close the card when its own close button is clicked
document.querySelectorAll(".close-btn").forEach(btn => {
  btn.addEventListener("click", () => {
    btn.parentElement.remove(); // remove the card that contains this button
```

```
  });
});
```

---

FOUNDATION 6

# Events — Making Things Interactive

Events are things that happen in the browser — a click, a key press, a scroll, a form submission, a mouse movement. JavaScript listens for these events and runs your code in response. This is the mechanism behind every interactive element on a website.

## addEventListener — The Core Pattern

```
// Syntax: element.addEventListener("eventType", callbackFunction)
const btn = document.querySelector(".book-btn");

btn.addEventListener("click", function() {
  console.log("Button was clicked!");
});

// Arrow function (same thing, cleaner)
btn.addEventListener("click", () => {
  console.log("Button clicked!");
});

// Named function (best when you need to remove the listener later)
function handleClick() {
  console.log("Clicked!");
}
btn.addEventListener("click", handleClick);
btn.removeEventListener("click", handleClick); // remove it later
```

## The Event Object

When an event fires, the browser automatically passes an **event object** to your callback. It contains information about what happened.

```
document.addEventListener("click", (e) => {
  console.log(e.type);       // "click"
  console.log(e.target);     // the exact element that was clicked
  console.log(e.clientX);    // X position of mouse
  console.log(e.clientY);    // Y position of mouse
});

// Prevent default browser behavior (e.g., stop form from submitting)
const form = document.querySelector("form");
form.addEventListener("submit", (e) => {
  e.preventDefault(); // stops the page from refreshing
```

```
  console.log("Form submitted via JS!");
});

// Stop event from bubbling up to parent elements
const card = document.querySelector(".card");
card.addEventListener("click", (e) => {
  e.stopPropagation(); // click won't reach parent listeners
});
```

## Common Event Types

| EVENT | WHEN IT FIRES | USED FOR |
|---|---|---|
| `click` | Mouse click | Buttons, links, cards |
| `mousemove` | Mouse moves over element | Custom cursor, 3D tilt, cursor gallery |
| `mouseenter` | Mouse enters element | Hover effects start |
| `mouseleave` | Mouse leaves element | Hover effects end |
| `scroll` | Page or element scrolls | Progress bar, nav transparency |
| `keydown` | Key pressed | ESC to close gallery, arrow nav |
| `keyup` | Key released | Search input filtering |
| `submit` | Form submitted | Booking forms |
| `input` | Input value changes | Live search, form validation |
| `change` | Select/checkbox changes | Date pickers, dropdowns |
| `load` | Page or resource loaded | Init after everything is ready |
| `DOMContentLoaded` | HTML parsed (before images) | Early JS init |
| `resize` | Window resized | Responsive JS, Three.js resize |
| `touchstart` | Touch begins | Mobile swipe detection |

## Practical Examples

```
// Mouse position (used for custom cursor and 3D tilt)
document.addEventListener("mousemove", (e) => {
  const x = e.clientX;                          // pixels from left
  const y = e.clientY;                          // pixels from top
  const nx = (x / window.innerWidth)  * 2 - 1; // -1 to +1
  const ny = (y / window.innerHeight) * 2 - 1; // -1 to +1
  console.log(nx, ny);
});

// Keyboard (close gallery on Escape)
document.addEventListener("keydown", (e) => {
  if (e.key === "Escape")      closeGallery();
  if (e.key === "ArrowRight")  nextImage();
  if (e.key === "ArrowLeft")   prevImage();
});
```

```
// Scroll (progress bar)
window.addEventListener("scroll", () => {
  const scrolled = window.scrollY;
  const total    = document.body.scrollHeight - window.innerHeight;
  const pct      = (scrolled / total) * 100;
  document.getElementById("progress").style.width = pct + "%";
}, { passive: true }); // passive: true = better scroll performance

// Window resize
window.addEventListener("resize", () => {
  camera.aspect = window.innerWidth / window.innerHeight; // Three.js
  camera.updateProjectionMatrix();
  renderer.setSize(window.innerWidth, window.innerHeight);
});
```

## Event Delegation — One Listener for Many Elements

Instead of adding a listener to 50 cards, add one listener to their parent. This is more efficient and also works for elements added dynamically.

```
// ✗ Bad — 50 separate listeners
document.querySelectorAll(".gallery-img").forEach(img => {
  img.addEventListener("click", openGallery);
});

// ✓ Good — one listener on the parent
document.querySelector(".gallery-strip").addEventListener("click", (e) => {
  const img = e.target.closest(".gallery-img"); // find clicked image
  if (img) openGallery(img);
});
```

> **Performance tip:** Add `{ passive: true }` to scroll and touch event listeners. It tells the browser you won't call `e.preventDefault()`, allowing it to scroll without waiting for your JS to finish — much smoother on mobile.

---

FOUNDATION 7

# Modern JavaScript (ES6+)

ES6 (also called ES2015) was a major update to JavaScript in 2015. Since then, new features are added every year. These modern features make code shorter, cleaner, and more readable. All libraries in this project use them extensively.

## Template Literals

```
const name    = "Dream Chalet";
const price   = 130;
const nights  = 5;
```

```
// Old way
const msg = "Welcome to " + name + ". Total: €" + (price * nights);

// Template literal — much cleaner
const msg = `Welcome to ${name}. Total: €${price * nights}`;

// Multi-line strings
const html = `
  <div class="card">
    <h3>${name}</h3>
    <p>€${price}/night</p>
  </div>
`;
```

## Optional Chaining (?.) and Nullish Coalescing (??)

```
const property = {
  name: "Dream Chalet",
  address: { city: "Trikala" }
};

// Without optional chaining — crashes if address is undefined
const country = property.address.country; // undefined (fine here)
const zip     = property.contact.phone;   // ✗ TypeError: Cannot read property of undefined

// With optional chaining — returns undefined instead of crashing
const phone = property?.contact?.phone;  // undefined (safe)
const city  = property?.address?.city;   // "Trikala"

// Nullish coalescing — default value when null/undefined
const lang    = localStorage.getItem("lang") ?? "el"; // "el" if null
const minStay = property.minStay ?? 2;               // 2 if undefined
```

## Short-Circuit Evaluation

```
// && — if left is falsy, return left; otherwise return right
const user = { name: "Maria", isAdmin: false };
user.isAdmin && showAdminPanel(); // showAdminPanel() only runs if isAdmin is true

// || — if left is truthy, return left; otherwise return right
const lang = savedLang || "el";   // use savedLang if truthy, else "el"

// Practical: conditionally add a class
const scrolled = window.scrollY > 80;
nav.classList.toggle("solid", scrolled); // second arg = force add/remove
```

## Spread and Rest

```
// Spread — expand iterable into individual items
const base  = ["WiFi", "Kitchen"];
const extra = ["Pool", "Fireplace"];
const all   = [...base, ...extra]; // ["WiFi", "Kitchen", "Pool", "Fireplace"]
```

```
// Spread to copy (avoids mutating original)
const original = { name: "Dream Chalet", rating: 9.8 };
const updated  = { ...original, rating: 9.9 }; // copy + override

// Rest — collect remaining arguments into an array
function logAll(first, ...others) {
  console.log("First:", first);
  console.log("Others:", others);
}
logAll("Meteora", "Elati", "Pertouli"); // Others: ["Elati", "Pertouli"]
```

## Modules — import / export

Modules let you split your code across multiple files and share code between them. Each file is a module.

```
// js/utils.js — export functions
export function calculateTotal(price, nights) {
  return price * nights;
}

export const PHONE = "+30 6976908838";

export default function greet(name) { // default export (one per file)
  return `Welcome, ${name}!`;
}

// js/main.js — import them
import greet, { calculateTotal, PHONE } from "./utils.js";

console.log(greet("Maria"));            // "Welcome, Maria!"
console.log(calculateTotal(130, 5));    // 650
console.log(PHONE);                     // "+30 6976908838"
```

```
<!-- In HTML, modules need type="module" -->
<script type="module" src="js/main.js"></script>
```

> **Note:** Modules only work over HTTP(S), not `file://` . Use a local server ( `npx serve .` ) when developing with modules.

---

FOUNDATION 8

# Async JavaScript & Debugging

JavaScript runs one line at a time. But some operations — like fetching data from a server, reading a file, or waiting for an image to load — take an unknown amount of time. If JavaScript waited for these, the page would freeze. **Asynchronous JavaScript** solves this by saying: "start this task, and when it finishes, run this function."

## The Problem — Synchronous Blocking

```
// If fetch() were synchronous (it's not), this would freeze the page
const data = fetch("https://api.example.com/data"); // ← page freezes here
console.log(data); // only runs after fetch completes
```

## Callbacks — The Old Way

```
// setTimeout — run code after a delay (non-blocking)
console.log("1. Start");

setTimeout(() => {
  console.log("3. After 2 seconds");
}, 2000);

console.log("2. This runs immediately");
// Output: "1. Start" → "2. This runs immediately" → "3. After 2 seconds"

// The callback (arrow function) runs LATER, when the timer fires
```

## Promises — A Better Way

```
// A Promise represents a value that will be available in the future
// It has 3 states: pending → fulfilled or rejected

const myPromise = new Promise((resolve, reject) => {
  // Simulate async work
  setTimeout(() => {
    const success = true;
    if (success) resolve("Data loaded!");  // success
    else reject("Something went wrong");   // failure
  }, 1000);
});

// Handle the result with .then() and .catch()
myPromise
  .then(result => console.log(result))  // "Data loaded!"
  .catch(error => console.error(error));

// fetch() returns a Promise
fetch("https://api.example.com/data")
  .then(response => response.json())    // parse JSON
  .then(data => console.log(data))      // use the data
  .catch(err => console.error(err));
```

## async / await — The Modern Way

async/await is syntactic sugar over Promises — same thing underneath, but written to look synchronous. Much easier to read and write.

```
// Any function marked "async" returns a Promise
async function loadTranslations(lang) {
  // "await" pauses execution INSIDE this function until Promise resolves
  // The rest of the page keeps running normally
  const response = await fetch(`/locales/${lang}.json`);
  const data     = await response.json();
  return data; // resolves the Promise with this value
}

// Calling an async function
async function init() {
  const translations = await loadTranslations("el");
  console.log(translations.hero.title); // "Dream Chalet"
}
init();

// Real example — fetch iCal feed
async function fetchAvailability() {
  const response = await fetch("/api/ical");
  if (!response.ok) throw new Error("Failed to load calendar");
  const text = await response.text(); // iCal is plain text
  return parseICal(text);
}
```

## Error Handling with try/catch

```
async function loadData(url) {
  try {
    const response = await fetch(url);
    if (!response.ok) {
      throw new Error(`HTTP error: ${response.status}`);
    }
    const data = await response.json();
    return data;

  } catch (error) {
    // Runs if fetch fails, JSON parse fails, or we threw manually
    console.error("Failed to load:", error.message);
    return null; // return a safe fallback

  } finally {
    // Always runs, whether success or failure
    hideLoadingSpinner();
  }
}

// Non-async: synchronous errors
try {
  const result = JSON.parse("invalid json {{{");
} catch (e) {
  console.error("Parse failed:", e.message);
}
```

## Promise.all — Run Multiple Async Tasks in Parallel

```
// ✗ Sequential — waits for each before starting the next (slow)
const airbnb  = await fetchCalendar(AIRBNB_URL);   // 1 second
const booking = await fetchCalendar(BOOKING_URL);  // + 1 second = 2 total

// ✓ Parallel — both start at the same time (fast)
const [airbnb, booking] = await Promise.all([
  fetchCalendar(AIRBNB_URL),
  fetchCalendar(BOOKING_URL)
]);
// Both complete in ~1 second total
```

## Debugging — Reading Errors and Using DevTools

Errors are normal. Every developer reads errors every day. Learning to read them quickly is a superpower.

```
// Most common errors and what they mean:

// ReferenceError: gsap is not defined
// → You forgot to load the GSAP script, or it loaded after your code

// TypeError: Cannot read properties of null (reading 'classList')
// → document.querySelector() returned null — element not found
// → Fix: check your CSS selector, make sure element exists in HTML

// SyntaxError: Unexpected token '}'
// → Typo in your code — a missing comma, bracket, or quote
// → Fix: look at the line number in the error message

// CORS error when fetching
// → You are fetching from a different domain without permission
// → Fix: use a proxy, or fetch server-side
```

```
// Debugging tools
console.log(variable);              // print a value
console.log({ variable });          // print with name: { variable: "value" }
console.table(arrayOfObjects);      // print as a table
console.group("Section name");      // group related logs
  console.log("inside group");
console.groupEnd();

// Debugger statement — pauses execution in DevTools
function calculatePrice(p, n) {
  debugger; // browser pauses here, you can inspect variables
  return p * n;
}

// Conditional debugging
if (price < 0) {
  console.error("Invalid price:", price);
  console.trace(); // print the call stack
}
```

## The 5-Step Debugging Process

1. **Read the error message** — it tells you the error type, message, and line number
2. **Go to that line** — open DevTools → Sources → find the file and line
3. **console.log everything** around that line — what values do your variables actually have?
4. **Check assumptions** — is the element actually in the DOM? Does the variable have the value you think?
5. **Google the exact error message** — someone has had this exact problem before. Stack Overflow is your friend.

> **You are now ready for the libraries.** Everything from Chapter 0 onwards builds on what you just learned — variables, functions, array methods, DOM selection, events, async/await. You will see all of it again in context, which is when it really clicks.

---

CHAPTER 0

# Loading Libraries from CDN

Before writing a single line of animation code, you need to understand how to load external libraries. A **CDN (Content Delivery Network)** is a server that hosts library files for you — no installation, no npm, no build tools. You just add a `<script>` tag to your HTML and the library is ready to use.

## The Basic Pattern

Libraries are loaded in the `<head>` (for CSS) or just before the closing `</body>` tag (for JavaScript). Order matters — load dependencies before the code that uses them.

```
<!DOCTYPE html>
<html lang="el">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Dream Chalet</title>

  <!-- CSS libraries go in <head> -->
  <link rel="stylesheet" href="https://unpkg.com/maplibre-gl/dist/maplibre-gl.css">
  <link rel="stylesheet" href="css/style.css">
</head>
<body>

  <!-- Your HTML content here -->

  <!-- JS libraries: load order matters! -->
  <!-- 1. GSAP core first -->
  <script src="https://cdn.jsdelivr.net/npm/gsap@3.12.5/dist/gsap.min.js"></script>
  <!-- 2. GSAP plugins after core -->
  <script src="https://cdn.jsdelivr.net/npm/gsap@3.12.5/dist/ScrollTrigger.min.js"></script>
  <script src="https://cdn.jsdelivr.net/npm/gsap@3.12.5/dist/Flip.min.js"></script>
  <script src="https://cdn.jsdelivr.net/npm/gsap@3.12.5/dist/Draggable.min.js"></script>
  <!-- 3. Other libraries -->
  <script src="https://unpkg.com/@studio-freight/lenis@1.0.42/dist/lenis.min.js"></script>
```

```
  <script src="https://cdn.jsdelivr.net/npm/three@0.160.0/build/three.min.js"></script>
  <script src="https://unpkg.com/maplibre-gl/dist/maplibre-gl.js"></script>
  <script src="https://cdn.jsdelivr.net/npm/i18next@23.7.6/dist/umd/i18next.min.js"></script>
  <!-- 4. Your own scripts LAST -->
  <script src="js/main.js"></script>
</body>
</html>
```

> **Why load JS before </body>?**
> If you put scripts in `<head>`, the browser loads and executes them before the HTML is rendered. That means your JavaScript can't find elements like `document.querySelector('.hero')` because they don't exist yet. Placing scripts at the bottom ensures all HTML is parsed first.

## Registering GSAP Plugins

GSAP plugins must be explicitly registered before use. This is a common beginner mistake — the plugin loads fine but animations don't work because registration was skipped.

```
// At the top of your main.js — always register plugins first
gsap.registerPlugin(ScrollTrigger, Flip, Draggable, SplitText);

// Now you can use them anywhere in your code
gsap.to(".hero", { opacity: 1, scrollTrigger: { trigger: ".hero" } });
```

> **Common mistake:** Forgetting `gsap.registerPlugin(ScrollTrigger)` and wondering why scroll animations do nothing. Always register all plugins you plan to use, right at the top of your main JS file.

---

CHAPTER 1

# JavaScript Essentials

Before diving into animation libraries, you need to be comfortable with the JavaScript patterns that appear everywhere in premium web projects. These are the building blocks everything else sits on.

## Selecting Elements

```
// Select ONE element (returns the first match)
const hero = document.querySelector('.hero');
const title = document.querySelector('#main-title');

// Select MULTIPLE elements (returns a NodeList)
const cards = document.querySelectorAll('.card');

// Loop over multiple elements
```

```
cards.forEach(card => {
  card.style.opacity = '0'; // hide all cards
});

// Convert NodeList to Array (for array methods like .map, .filter)
const cardsArray = Array.from(document.querySelectorAll('.card'));
```

## Event Listeners

```
// Basic click event
const btn = document.querySelector('.book-btn');
btn.addEventListener('click', () => {
  console.log('Button clicked!');
});

// Mouse position tracking (used for cursor gallery + 3D tilt)
document.addEventListener('mousemove', (e) => {
  const x = e.clientX; // pixels from left edge
  const y = e.clientY; // pixels from top edge

  // Normalize to -1 → +1 range (useful for 3D rotation)
  const nx = (x / window.innerWidth)  * 2 - 1;
  const ny = (y / window.innerHeight) * 2 - 1;
});

// Scroll event (but prefer ScrollTrigger for animations)
window.addEventListener('scroll', () => {
  const scrollY = window.scrollY; // how far page has scrolled
});

// Keyboard events
document.addEventListener('keydown', (e) => {
  if (e.key === 'Escape') closeGallery();
  if (e.key === 'ArrowRight') nextImage();
});
```

## Async / Await and Fetch

Used to load data from external sources — like fetching an iCal feed or JSON translation file.

```
// Fetch is how you load data from a URL
async function loadTranslations(lang) {
  try {
    const response = await fetch(`/locales/${lang}.json`);
    if (!response.ok) throw new Error('Failed to load translations');
    const data = await response.json(); // parse JSON
    return data;
  } catch (error) {
    console.error('Translation load error:', error);
    return {}; // return empty object as fallback
  }
}

// Using it:
```

```
const translations = await loadTranslations('el');
console.log(translations.hero.title); // "Dream Chalet"
```

## IntersectionObserver

Tells you when an element enters or leaves the viewport. Used to pause Three.js animations when off-screen (saves battery and performance).

```
// Create an observer
const observer = new IntersectionObserver((entries) => {
  entries.forEach(entry => {
    if (entry.isIntersecting) {
      // Element is now visible — start animation
      startThreeJsAnimation();
    } else {
      // Element left the viewport — stop animation
      stopThreeJsAnimation();
    }
  });
}, {
  threshold: 0.1 // trigger when 10% of element is visible
});

// Tell it which element to watch
const canvas = document.querySelector('#three-canvas');
observer.observe(canvas);
```

## requestAnimationFrame

The browser's built-in animation loop. It runs your function ~60 times per second, perfectly synced with screen refresh. This is how Three.js and smooth animations work.

```
let animating = true;

function animate() {
  if (!animating) return; // stop the loop

  // Update your animation here
  cube.rotation.y += 0.01;
  renderer.render(scene, camera);

  requestAnimationFrame(animate); // schedule next frame
}

animate(); // kick off the loop
```

## localStorage

Saves small pieces of data in the browser that persist between page visits. Used to remember the user's language preference.

```
// Save a value (always saved as a string)
localStorage.setItem('lang', 'el');
localStorage.setItem('theme', 'dark');

// Read it back
const lang = localStorage.getItem('lang'); // 'el'

// Remove it
localStorage.removeItem('lang');

// Practical pattern: load with fallback
const savedLang = localStorage.getItem('lang') || 'el'; // default Greek
```

> **Tip:** Use the browser DevTools (F12) → Application → Local Storage to see and clear what's stored.

## CSS Custom Properties (Variables) via JS

```
// Read a CSS variable
const gold = getComputedStyle(document.documentElement)
  .getPropertyValue('--gold'); // '#c9a96e'

// Set a CSS variable dynamically (affects all elements using it)
document.documentElement.style.setProperty('--gold', '#e8c97e');

// Useful for theming — change one variable, entire site updates
```

---

CHAPTER 2

# GSAP Core

**GSAP (GreenSock Animation Platform)** is the industry-standard JavaScript animation library. It animates anything — CSS properties, SVG, canvas, DOM attributes — with buttery-smooth performance across all browsers. Think of it as a remote control for every visual property on your page.

> **Why GSAP for Dream Chalet?**
> CSS animations are limited. GSAP lets us animate letter-by-letter text reveals, synchronize a dozen elements in a precise sequence, create magnetic button effects, and connect animations directly to scroll position — all without performance issues.

## gsap.to() — Animate TO a value

The most common method. The element starts at its current state and animates *to* the values you specify.

```
// Move an element 100px to the right over 1 second
gsap.to(".box", { x: 100, duration: 1 });

// Fade in the hero title
gsap.to(".hero-title", { opacity: 1, duration: 1.2, ease: "power3.out" });

// Animate multiple properties at once
gsap.to(".nav", {
  backgroundColor: "#0e0b08",
  padding: "12px 2rem",
  duration: 0.4,
  ease: "power2.out"
});
```

## gsap.from() — Animate FROM a value

The element starts at the values you specify and animates back to its natural CSS state. Perfect for entrance animations.

```
// Element slides up from 60px below and fades in
gsap.from(".section-title", {
  y: 60,
  opacity: 0,
  duration: 1,
  ease: "power3.out"
});

// Multiple elements, staggered (each 0.1s apart)
gsap.from(".amenity-card", {
  y: 40,
  opacity: 0,
  duration: 0.8,
  stagger: 0.1,  // 0.1s delay between each card
  ease: "power2.out"
});
```

## gsap.fromTo() — Full control

You define both the start AND end state explicitly. Use when you need precise control.

```
gsap.fromTo(".hero-title",
  // FROM (starting state)
  { opacity: 0, y: 80, letterSpacing: "0.5em" },
  // TO (ending state)
  { opacity: 1, y: 0, letterSpacing: "0.08em", duration: 1.4, ease: "power4.out" }
);
```

## Easing Reference

Easing controls the *feel* of the animation — how it accelerates and decelerates. This is what separates professional animations from robotic ones.

| EASE | FEEL | BEST FOR |
|---|---|---|
| `power1.out` | Subtle deceleration | Small UI elements |
| `power2.out` | Smooth deceleration | Buttons, cards |
| `power3.out` | Strong deceleration | Headings, panels |
| `power4.out` | Very strong snap | Hero text reveals |
| `expo.out` | Explosive then stop | Menu opens |
| `elastic.out(1, 0.3)` | Bouncy spring | Playful UI |
| `back.out(1.7)` | Slight overshoot | Scale-in buttons |
| `none / linear` | Constant speed | Scroll scrubbing |

## Targeting Elements

```
// CSS selector string — targets ALL matching elements
gsap.to(".card", { opacity: 0 });

// DOM element reference
const hero = document.querySelector(".hero");
gsap.to(hero, { scale: 1.05 });

// Array of elements
const elements = document.querySelectorAll(".card");
gsap.to(elements, { y: -20, stagger: 0.05 });

// GSAP selector shorthand (scoped to a container)
const ctx = gsap.context(() => {
  gsap.from("h1", { opacity: 0 }); // only h1 inside .section
}, ".section");
```

## Common CSS Properties in GSAP

| GSAP PROPERTY | CSS EQUIVALENT | EXAMPLE |
|---|---|---|
| `x` | translateX | `x: 100` → moves right 100px |
| `y` | translateY | `y: -50` → moves up 50px |
| `scale` | scale() | `scale: 1.2` → 20% bigger |
| `rotation` | rotate() | `rotation: 45` → 45 degrees |
| `opacity` | opacity | `opacity: 0` → invisible |
| `width` | width | `width: "100%"` |
| `backgroundColor` | background-color | `backgroundColor: "#fff"` |
| `clipPath` | clip-path | `clipPath: "inset(0 0 0 100%)"` |

## Callbacks

```
gsap.to(".box", {
  x: 200,
  duration: 1,
  onStart: () => console.log("Animation started!"),
  onComplete: () => console.log("Animation done!"),
  onUpdate: () => console.log("Animating..."), // called every frame
});
```

---

CHAPTER 3

# GSAP Timelines

A **Timeline** is a sequence of animations played in order. Instead of using `delay` to chain animations (fragile and hard to maintain), a Timeline lets you build complex multi-step sequences that are easy to read, adjust, and control.

## Creating a Timeline

```
// Create a timeline
const tl = gsap.timeline();

// Add animations in sequence — each starts after the previous finishes
tl.from(".nav", { y: -80, opacity: 0, duration: 0.8 })
  .from(".hero-title", { y: 60, opacity: 0, duration: 1, ease: "power3.out" })
  .from(".hero-subtitle", { y: 40, opacity: 0, duration: 0.8 }, "-=0.4")
  // "-=0.4" means start 0.4 seconds BEFORE the previous ends (overlap)
  .from(".hero-btns", { y: 30, opacity: 0, duration: 0.6, stagger: 0.15 }, "-=0.3");
```

## Position Parameter

```
const tl = gsap.timeline();

tl.from(".a", { opacity: 0, duration: 1 })        // at time 0
  .from(".b", { opacity: 0, duration: 1 })        // at time 1 (after .a)
  .from(".c", { opacity: 0, duration: 1 }, "+=0.5") // 0.5s AFTER .b ends
  .from(".d", { opacity: 0, duration: 1 }, "-=0.3") // 0.3s BEFORE .c ends
  .from(".e", { opacity: 0, duration: 1 }, "<")     // SAME TIME as .d
  .from(".f", { opacity: 0, duration: 1 }, 2);      // exactly at 2 seconds
```

## Timeline Defaults

```
// Set defaults for all animations in this timeline
const tl = gsap.timeline({
  defaults: {
    duration: 0.8,
    ease: "power2.out"
```

```
  }
});

// No need to repeat duration/ease on each animation
tl.from(".title", { y: 50, opacity: 0 })
  .from(".subtitle", { y: 30, opacity: 0 })
  .from(".btn", { scale: 0.9, opacity: 0 });
```

## Controlling Timelines

```
const tl = gsap.timeline({ paused: true }); // start paused
tl.from(".menu-item", { x: -40, opacity: 0, stagger: 0.08 });

// Control it programmatically
document.querySelector(".menu-open").addEventListener("click", () => tl.play());
document.querySelector(".menu-close").addEventListener("click", () => tl.reverse());

// Other controls
tl.pause();          // freeze
tl.resume();         // continue
tl.restart();        // go back to start and play
tl.seek(0.5);        // jump to 0.5 seconds
tl.timeScale(2);     // play at 2x speed
tl.timeScale(0.5);   // play in slow motion
```

> **Pro tip:** Use `paused: true` and trigger `tl.play()` on user interaction (click, scroll entry). This makes your animations intentional rather than playing on load automatically.

---

CHAPTER 4

# GSAP ScrollTrigger

ScrollTrigger connects animations to scroll position. It can trigger animations when elements enter the viewport, pin sections in place while scrolling, and even scrub (directly tie) animation progress to scroll position.

> **Used in Dream Chalet for:** Transparent nav → solid on scroll, all section entrance animations, pinned scroll-scrubbed video, pinned horizontal attractions section, gallery Draggable strip.

## Basic Trigger — Animate on Scroll Enter

```
// Always register ScrollTrigger first!
gsap.registerPlugin(ScrollTrigger);

// Animate when element enters the viewport
gsap.from(".section-title", {
```

```
  y: 60,
  opacity: 0,
  duration: 1,
  ease: "power3.out",
  scrollTrigger: {
    trigger: ".section-title", // which element triggers it
    start: "top 80%",          // when top of element hits 80% from top of viewport
    // toggleActions: "onEnter onLeave onEnterBack onLeaveBack"
    toggleActions: "play none none none"  // play once, never reverse
  }
});
```

## start / end Values

```
// Format: "triggerPosition viewportPosition"
// Positions: "top", "center", "bottom", or a pixel/percentage value

scrollTrigger: {
  start: "top 80%",     // trigger's top hits 80% down the viewport
  start: "center center", // trigger's center hits viewport center
  start: "top top",     // trigger's top hits the very top of viewport
  start: "top+=100 80%", // 100px below trigger's top hits 80% viewport
  end: "bottom top",    // trigger's bottom hits viewport top
  end: "+=500",         // 500px after the start point
}
```

## Pinning — Keep Element Fixed While Scrolling

```
// Pin the scroll-video section while the video plays
ScrollTrigger.create({
  trigger: ".scroll-video-section",
  start: "top top",
  end: "+=2000",   // pin for 2000px of scroll
  pin: true,       // pin the trigger element
  anticipatePin: 1 // prevents jump on mobile
});
```

## Scrubbing — Tie Animation to Scroll

```
// Video currentTime follows scroll position exactly
const video = document.querySelector(".scroll-video");

gsap.to(video, {
  currentTime: video.duration, // animate from 0 to full duration
  ease: "none",                // linear — no easing on scrub
  scrollTrigger: {
    trigger: ".scroll-video-section",
    start: "top top",
    end: "bottom bottom",
    scrub: true,   // true = direct tie to scroll
    // scrub: 1    // 1 = 1 second lag (smoother feel)
    pin: true
```

```
  }
});
```

## Horizontal Scroll (Attractions Section)

```
const cards = gsap.utils.toArray(".attraction-card");

gsap.to(cards, {
  xPercent: -100 * (cards.length - 1), // slide all cards left
  ease: "none",
  scrollTrigger: {
    trigger: ".attractions-wrapper",
    pin: true,
    scrub: 1,
    start: "top top",
    end: () => `+=${cards.length * window.innerWidth}`,
    snap: {
      snapTo: 1 / (cards.length - 1), // snap to each card
      duration: { min: 0.2, max: 0.5 },
      ease: "power2.inOut"
    }
  }
});
```

## Navbar Transparency Toggle

```
// Add/remove class based on scroll position
ScrollTrigger.create({
  start: "top -80",  // 80px after page top
  end: 99999,
  toggleClass: { targets: ".nav", className: "scrolled" }
});

// In CSS:
// .nav { background: transparent; transition: background 0.3s; }
// .nav.scrolled { background: rgba(14, 11, 8, 0.95); backdrop-filter: blur(10px); }
```

## Stagger Entrance for Multiple Elements

```
// Animate all amenity cards with stagger on scroll
gsap.from(".amenity-item", {
  y: 50,
  opacity: 0,
  duration: 0.7,
  stagger: 0.1,
  ease: "power2.out",
  scrollTrigger: {
    trigger: ".amenities-grid",
    start: "top 75%"
  }
});
```

> **Debugging tip:** Add `markers: true` to any ScrollTrigger to see start/end markers in the browser. Remove before going live.

```
scrollTrigger: { trigger: ".hero", start: "top 80%", markers: true }
```

---

CHAPTER 5

# GSAP Flip Plugin

Flip solves one of the hardest animation problems: animating an element that *moves in the DOM*. When a photo moves from a thumbnail in a grid to a fullscreen display, it changes parent, position, and size all at once. Without Flip, this looks like a teleport. With Flip, it appears to *physically travel* across the screen.

> **Used in Dream Chalet for:** The gallery expansion — click a photo in the draggable strip, it morphs from its strip position to fill the screen. Click close, it morphs back. No jarring jumps.

## How Flip Works (the concept)

Flip uses a technique called FLIP (First, Last, Invert, Play):

1. **First** — record the element's current position/size
2. **Last** — move it to its new position in the DOM
3. **Invert** — instantly offset it back to the First position
4. **Play** — animate it forward to the Last position

The result: the element appears to glide smoothly from A to B.

## Gallery Expansion with Flip

```
gsap.registerPlugin(Flip);

const strip = document.querySelector(".gallery-strip");
const fullscreen = document.querySelector(".gallery-fullscreen");

function openImage(img) {
  // Step 1: Record the CURRENT state (position in the strip)
  const state = Flip.getState(img);

  // Step 2: Move the image to the fullscreen container
  fullscreen.appendChild(img);
  fullscreen.classList.add("active"); // show the fullscreen overlay

  // Step 3: Animate FROM the recorded old position TO the new position
```

```
  Flip.from(state, {
    duration: 0.7,
    ease: "power2.inOut",
    absolute: true,    // use absolute positioning during animation
    onComplete: () => img.classList.add("fullscreen-active")
  });
}

function closeImage(img) {
  const state = Flip.getState(img);

  // Move back to the strip
  strip.appendChild(img);
  fullscreen.classList.remove("active");

  Flip.from(state, {
    duration: 0.6,
    ease: "power2.inOut",
    absolute: true
  });
}

// Attach events
document.querySelectorAll(".gallery-img").forEach(img => {
  img.addEventListener("click", () => openImage(img));
});
document.querySelector(".close-btn").addEventListener("click", () => {
  const activeImg = fullscreen.querySelector("img");
  if (activeImg) closeImage(activeImg);
});
```

## Flip with Layout Changes

```
// Animating a grid reorder (e.g., filtering items)
const items = document.querySelectorAll(".grid-item");

filterBtn.addEventListener("click", () => {
  // Record positions BEFORE the change
  const state = Flip.getState(items);

  // Make the DOM change (filter, reorder, etc.)
  items.forEach(item => {
    item.style.display = item.dataset.category === "bedroom" ? "block" : "none";
  });

  // Animate FROM old positions TO new positions
  Flip.from(state, {
    duration: 0.5,
    ease: "power2.inOut",
    stagger: 0.05
  });
});
```

---

CHAPTER 6

# GSAP Draggable + InertiaPlugin

**Draggable** makes any element draggable with a few lines of code. When combined with **InertiaPlugin**, dragged elements continue moving after release — like flicking a card on a table — giving the interaction a natural, physical feel.

> **Used in Dream Chalet for:** The gallery horizontal strip. Users drag through photos with momentum, making browsing feel tactile and premium instead of clicking arrows.

## Basic Draggable

```
gsap.registerPlugin(Draggable);

// Make an element draggable horizontally only
Draggable.create(".gallery-strip", {
  type: "x",          // "x" = horizontal only, "y" = vertical, "xy" = both
  bounds: ".gallery-wrapper",  // constrain to parent bounds
  onDrag: function() {
    console.log("Current x:", this.x);
  },
  onDragEnd: function() {
    console.log("Final x:", this.x);
  }
});
```

## Adding Momentum with InertiaPlugin

```
// InertiaPlugin is included with GSAP Club (free with account)
// Load it from CDN:
// <script src="https://cdn.jsdelivr.net/npm/gsap@3.12.5/dist/InertiaPlugin.min.js"></script>

gsap.registerPlugin(Draggable, InertiaPlugin);

Draggable.create(".gallery-strip", {
  type: "x",
  bounds: ".gallery-wrapper",
  inertia: true,         // enable momentum on release
  // edgeResistance: 0.85, // resistance when dragging past bounds (0-1)
  onThrowComplete: function() {
    // called when momentum animation finishes
    console.log("Settled at:", this.x);
  }
});
```

## Full Gallery Strip Implementation

```
const wrapper = document.querySelector(".gallery-wrapper");
const strip = document.querySelector(".gallery-strip");

// Calculate the maximum drag distance
function getMaxDrag() {
  return -(strip.scrollWidth - wrapper.offsetWidth);
}

Draggable.create(strip, {
  type: "x",
  inertia: true,
  bounds: { minX: getMaxDrag(), maxX: 0 },
  edgeResistance: 0.85,

  // Visual feedback while dragging
  onDragStart: function() {
    strip.style.cursor = "grabbing";
  },
  onDragEnd: function() {
    strip.style.cursor = "grab";
  }
});

// Recalculate bounds on window resize
window.addEventListener("resize", () => {
  Draggable.get(strip).applyBounds({ minX: getMaxDrag(), maxX: 0 });
});
```

## Magnetic Button Effect

This is not a Draggable feature — it uses GSAP's `quickTo()` for extremely performant cursor tracking. The button "attracts" the cursor when it gets close.

```
document.querySelectorAll(".magnetic-btn").forEach(btn => {
  // quickTo creates a cached, optimized setter — much faster than gsap.to() in a loop
  const xTo = gsap.quickTo(btn, "x", { duration: 0.4, ease: "power2.out" });
  const yTo = gsap.quickTo(btn, "y", { duration: 0.4, ease: "power2.out" });

  btn.addEventListener("mousemove", (e) => {
    const rect = btn.getBoundingClientRect();
    // Distance from cursor to button center
    const dx = e.clientX - (rect.left + rect.width / 2);
    const dy = e.clientY - (rect.top + rect.height / 2);
    // Move button 30% toward the cursor
    xTo(dx * 0.3);
    yTo(dy * 0.3);
  });

  btn.addEventListener("mouseleave", () => {
    // Snap back to original position
    xTo(0);
    yTo(0);
  });
});
```

> **Tip:** Apply the magnetic effect to the contact section buttons (WhatsApp, Viber, Airbnb, Booking.com). It makes them feel alive and invites interaction — exactly what you want for your booking CTAs.

---

CHAPTER 7

# GSAP SplitText

SplitText breaks a text element into individual characters, words, or lines — each wrapped in its own `<span>` — so you can animate them independently. This is how you create the dramatic letter-by-letter title reveals seen on premium hospitality sites.

> **Used in Dream Chalet for:** The hero title "Dream Chalet" letter-by-letter reveal on load, and the attraction card titles revealing as each card enters during horizontal scroll.

## Basic SplitText

```
// Load SplitText (GSAP Club — free with account)
// <script src="https://cdn.jsdelivr.net/npm/gsap@3.12.5/dist/SplitText.min.js"></script>

gsap.registerPlugin(SplitText);

// Split the title into individual characters
const split = new SplitText(".hero-title", { type: "chars" });

// Now animate each character
gsap.from(split.chars, {
  opacity: 0,
  y: 80,
  rotationX: -90,   // 3D flip effect
  duration: 0.8,
  stagger: 0.03,    // 30ms between each character
  ease: "power4.out",
  transformOrigin: "0% 50% -50px"
});
```

## Split Types

```
// Split into characters
const byChars = new SplitText(".title", { type: "chars" });
// → split.chars = [span, span, span, ...] one per character

// Split into words
const byWords = new SplitText(".subtitle", { type: "words" });
// → split.words = [span, span, ...] one per word
```

```
// Split into lines (based on rendered line breaks)
const byLines = new SplitText(".paragraph", { type: "lines" });
// → split.lines = [span, span, ...] one per line

// Split into all three at once
const all = new SplitText(".text", { type: "chars,words,lines" });
```

## Line Reveal (Mask Effect)

A popular premium effect: each line slides up from behind an invisible mask, hiding the animation start point.

```
const split = new SplitText(".about-text", { type: "lines" });

// Wrap each line in a div with overflow:hidden to create the mask
split.lines.forEach(line => {
  const wrapper = document.createElement("div");
  wrapper.style.overflow = "hidden";
  line.parentNode.insertBefore(wrapper, line);
  wrapper.appendChild(line);
});

// Now animate lines from below — they appear to "emerge" from behind the mask
gsap.from(split.lines, {
  y: "100%",
  opacity: 0,
  duration: 0.9,
  stagger: 0.12,
  ease: "power3.out",
  scrollTrigger: {
    trigger: ".about-text",
    start: "top 80%"
  }
});
```

## Revert SplitText

```
// After the animation, restore the original HTML (important for accessibility)
const split = new SplitText(".title", { type: "chars" });
const anim = gsap.from(split.chars, { opacity: 0, y: 50, stagger: 0.05 });

anim.then(() => {
  split.revert(); // restores original text structure
});
```

> **Accessibility note:** SplitText wraps each character in a span, which can confuse screen readers. Add `aria-label="Dream Chalet"` to the parent element so assistive technology reads the full text correctly.

---

CHAPTER 8

# Lenis Smooth Scroll

Lenis replaces the browser's default abrupt scroll with a smooth, momentum-based scrolling experience. When the user scrolls, instead of snapping to the new position, the page glides there with a natural ease — like scrolling on iOS. This alone makes a site feel dramatically more premium.

> **Used in Dream Chalet for:** The entire page scroll experience. Without Lenis, even the best GSAP animations can feel slightly jarring. With Lenis, everything flows as one cohesive experience.

## Basic Setup

```
// Create Lenis instance
const lenis = new Lenis({
  duration: 1.2,         // scroll animation duration (seconds)
  easing: (t) => Math.min(1, 1.001 - Math.pow(2, -10 * t)), // expo ease
  orientation: "vertical",
  smoothWheel: true,     // smooth mouse wheel
  syncTouch: false,      // don't override native touch scroll (iOS)
});

// The animation loop — Lenis needs to be updated every frame
function raf(time) {
  lenis.raf(time);
  requestAnimationFrame(raf);
}
requestAnimationFrame(raf);
```

## Connecting Lenis to GSAP ScrollTrigger

This is critical. Without this connection, ScrollTrigger uses the native scroll position, which conflicts with Lenis's virtual scroll. You MUST add these lines.

```
const lenis = new Lenis({ duration: 1.2 });

// Tell GSAP ticker to update Lenis on each frame
gsap.ticker.add((time) => {
  lenis.raf(time * 1000); // GSAP time is in seconds, Lenis needs milliseconds
});

// Tell ScrollTrigger to use Lenis scroll values
lenis.on("scroll", ScrollTrigger.update);

// Disable GSAP's default lag smoothing (Lenis handles this)
gsap.ticker.lagSmoothing(0);
```

## Programmatic Scroll

```
// Scroll to a target element smoothly
lenis.scrollTo(".contact-section", {
  offset: -80,    // -80px offset (accounts for fixed nav height)
  duration: 1.5,
  easing: (t) => t < 0.5 ? 2*t*t : -1+(4-2*t)*t  // ease in-out
});

// Scroll to a specific pixel position
lenis.scrollTo(2400);

// Scroll to top
lenis.scrollTo(0, { duration: 1.2 });

// Use this for nav link clicks:
document.querySelectorAll("nav a").forEach(link => {
  link.addEventListener("click", (e) => {
    e.preventDefault();
    const target = document.querySelector(link.getAttribute("href"));
    lenis.scrollTo(target, { offset: -80 });
  });
});
```

## Stop / Start (for modals)

```
// When gallery fullscreen opens — stop background scroll
function openGallery() {
  lenis.stop();
  // ... open gallery
}

// When gallery closes — resume scroll
function closeGallery() {
  lenis.start();
  // ... close gallery
}
```

---

CHAPTER 9

# Three.js

Three.js is a JavaScript library that makes WebGL accessible. WebGL is a browser API that lets you draw 3D graphics using the GPU — but it's extremely low-level and complex to use directly. Three.js wraps it in a friendly API, so you can create 3D scenes, lights, cameras, and objects without needing to understand GPU shaders.

Think of it like this: WebGL is the engine, Three.js is the car.

> **Used in Dream Chalet for:** A floating golden particle field rendered on a `<canvas>` element behind the About section and Attractions section — adds depth, luxury, and visual interest without heavy content.

## The Core Concepts

| CONCEPT | REAL-WORLD ANALOGY | WHAT IT DOES |
|---|---|---|
| Scene | A film set | Container that holds everything — objects, lights, camera |
| Camera | The film camera | Defines what we see and from what angle |
| Renderer | The film projector | Draws the scene onto a <canvas> element |
| Geometry | A wire sculpture | The shape of a 3D object (box, sphere, plane...) |
| Material | Paint or texture | The surface appearance (color, shininess, texture...) |
| Mesh | The finished sculpture | Geometry + Material combined = a visible 3D object |
| Points | A star field | Geometry rendered as individual dots — used for particles |

## Minimal Scene Setup

```
// 1. Scene — the container
const scene = new THREE.Scene();

// 2. Camera — PerspectiveCamera(fov, aspectRatio, near, far)
const camera = new THREE.PerspectiveCamera(
  75,                                  // field of view in degrees
  window.innerWidth / window.innerHeight, // aspect ratio
  0.1,                                 // near clipping plane
  1000                                 // far clipping plane
);
camera.position.z = 5; // move camera back so we can see the scene

// 3. Renderer — draws to a canvas
const renderer = new THREE.WebGLRenderer({
  canvas: document.querySelector("#three-canvas"),
  alpha: true,        // transparent background (show page behind it)
  antialias: true     // smooth edges
});
renderer.setSize(window.innerWidth, window.innerHeight);
renderer.setPixelRatio(Math.min(window.devicePixelRatio, 2)); // max 2x for performance

// 4. Create a simple cube
const geometry = new THREE.BoxGeometry(1, 1, 1);
const material = new THREE.MeshBasicMaterial({ color: 0xc9a96e });
const cube = new THREE.Mesh(geometry, material);
scene.add(cube);

// 5. Animation loop
function animate() {
  requestAnimationFrame(animate);
  cube.rotation.y += 0.01;
  renderer.render(scene, camera);
```

```
}
animate();
```

## Creating a Particle System (Dream Chalet style)

```
// Particle system — 2000 floating golden dots
const particleCount = 2000;

// BufferGeometry is efficient for large numbers of points
const geometry = new THREE.BufferGeometry();

// Create random positions for each particle (x, y, z)
const positions = new Float32Array(particleCount * 3);
for (let i = 0; i < particleCount * 3; i++) {
  positions[i] = (Math.random() - 0.5) * 20; // random between -10 and +10
}
geometry.setAttribute("position", new THREE.BufferAttribute(positions, 3));

// PointsMaterial — how each particle looks
const material = new THREE.PointsMaterial({
  color: 0xc9a96e,   // warm gold
  size: 0.04,        // particle size
  transparent: true,
  opacity: 0.7,
  sizeAttenuation: true  // particles farther away appear smaller
});

// Points = Geometry + Material (like Mesh but for particles)
const particles = new THREE.Points(geometry, material);
scene.add(particles);

// Animate — slowly rotate the entire particle field
function animate() {
  requestAnimationFrame(animate);
  particles.rotation.y += 0.0003;
  particles.rotation.x += 0.0001;
  renderer.render(scene, camera);
}
animate();
```

## Handle Window Resize

```
window.addEventListener("resize", () => {
  // Update camera aspect ratio
  camera.aspect = window.innerWidth / window.innerHeight;
  camera.updateProjectionMatrix();

  // Update renderer size
  renderer.setSize(window.innerWidth, window.innerHeight);
  renderer.setPixelRatio(Math.min(window.devicePixelRatio, 2));
});
```

## Pause When Off-Screen (Performance)

```
let isVisible = false;

const observer = new IntersectionObserver(([entry]) => {
  isVisible = entry.isIntersecting;
});
observer.observe(document.querySelector("#three-canvas"));

function animate() {
  requestAnimationFrame(animate);
  if (!isVisible) return; // skip rendering when canvas is not visible
  particles.rotation.y += 0.0003;
  renderer.render(scene, camera);
}
animate();
```

## Disable on Low-End Devices

```
// Skip Three.js entirely on weak devices (saves battery on mobile)
const isLowEnd = navigator.hardwareConcurrency < 4 ||
                 /Mobi|Android/i.test(navigator.userAgent);

if (!isLowEnd) {
  initThreeJS(); // only init on capable devices
}
```

> **Performance rule:** Keep particle count under 5000 for reliable 60fps. Use `renderer.setPixelRatio(Math.min(window.devicePixelRatio, 2))` — never let it go above 2, or retina screens will render at 3–4x resolution and tank performance.

---

CHAPTER 10

# MapLibre GL JS

**MapLibre GL JS** is a free, open-source map rendering library — a fork of Mapbox GL JS v1. It renders interactive, vector-based maps that look smooth at any zoom level, support custom styling, and allow you to add custom markers, popups, and animations. *No API key. No account. No cost.*

> **Used in Dream Chalet for:** An interactive map showing the property location + 4 nearby attractions (Meteora, Elati, Pertouli, Palaiokarya). Clicking an attraction card on the page flies the map camera to that location.

## Basic Map Setup

```
<!-- In your HTML -->
<link rel="stylesheet" href="https://unpkg.com/maplibre-gl/dist/maplibre-gl.css">
<script src="https://unpkg.com/maplibre-gl/dist/maplibre-gl.js"></script>

<div id="map" style="width: 100%; height: 500px;"></div>
```

```
const map = new maplibregl.Map({
  container: "map",   // the div id
  // Free tile source — no API key needed
  style: "https://tiles.openfreemap.org/styles/positron",
  center: [21.75, 39.61], // [longitude, latitude] — Trikala, Greece
  zoom: 10,
  pitch: 30,          // tilt the map (0 = flat, 60 = very tilted)
  bearing: 0          // rotation in degrees
});
```

## Adding Custom Markers

```
// Define all locations
const locations = [
  {
    name: "Dream Chalet",
    coords: [21.75, 39.61],
    icon: "🏠",
    color: "#c9a96e",   // gold
    description: "Your home for the stay",
    isMain: true
  },
  {
    name: "Meteora",
    coords: [21.6311, 39.7217],
    icon: "⛪",
    color: "#8a7a6a",
    description: "UNESCO World Heritage — 22 km",
    isMain: false
  },
  {
    name: "Elati",
    coords: [21.5833, 39.5333],
    icon: "⛰️",
    color: "#8a7a6a",
    description: "Mountain Village — 30 km",
    isMain: false
  },
  {
    name: "Palaiokarya Waterfalls",
    coords: [21.5167, 39.5333],
    icon: "💧",
    color: "#8a7a6a",
    description: "Hidden Gem — 35 km",
    isMain: false
  },
  {
    name: "Pertouli",
    coords: [21.5833, 39.4833],
    icon: "⛷️",
```

```
    color: "#8a7a6a",
    description: "Ski Village — 54 km",
    isMain: false
  }
];

// Add all markers to the map
locations.forEach(loc => {
  // Create custom marker element
  const el = document.createElement("div");
  el.className = "custom-marker";
  el.innerHTML = loc.icon;
  el.style.cssText = `
    font-size: ${loc.isMain ? "28px" : "20px"};
    cursor: pointer;
    filter: drop-shadow(0 2px 4px rgba(0,0,0,0.4));
    transition: transform 0.2s;
  `;
  el.addEventListener("mouseenter", () => el.style.transform = "scale(1.3)");
  el.addEventListener("mouseleave", () => el.style.transform = "scale(1)");

  // Create popup
  const popup = new maplibregl.Popup({ offset: 25, closeButton: false })
    .setHTML(`
      <strong>${loc.name}</strong>
      <p>${loc.description}</p>
    `);

  // Add marker to map
  new maplibregl.Marker({ element: el })
    .setLngLat(loc.coords)
    .setPopup(popup)
    .addTo(map);
});
```

## Flying to a Location (connected to Attractions section)

```
// When user clicks an attraction card, fly the map there
document.querySelectorAll(".attraction-card").forEach(card => {
  card.addEventListener("click", () => {
    const name = card.dataset.location; // e.g., "meteora"
    const loc = locations.find(l => l.name.toLowerCase().includes(name));

    if (loc) {
      map.flyTo({
        center: loc.coords,
        zoom: 12,
        pitch: 45,
        bearing: 15,
        duration: 2000,      // animation duration in ms
        essential: true      // not affected by prefers-reduced-motion
      });
    }
  });
});
```

## Custom Map Style (warm/dark theme)

```
// Available free styles (no API key):
// Light:   "https://tiles.openfreemap.org/styles/positron"
// Dark:    "https://tiles.openfreemap.org/styles/dark-matter"
// Natural: "https://tiles.openfreemap.org/styles/bright"

// Or use MapTiler free styles:
// "https://api.maptiler.com/maps/streets/style.json?key=YOUR_FREE_KEY"

// For Dream Chalet, use dark-matter and style markers with gold color
const map = new maplibregl.Map({
  container: "map",
  style: "https://tiles.openfreemap.org/styles/dark-matter",
  center: [21.75, 39.61],
  zoom: 9
});
```

> **Tip:** MapLibre renders maps using WebGL — the same technology as Three.js. Both need a canvas and the GPU, so having both active simultaneously can tax mobile devices. Pause the Three.js particle animation when the map section is visible using IntersectionObserver.

---

CHAPTER 11

# i18next — Multilingual Websites

**i18next** is the most widely-used JavaScript internationalization (i18n) library. It lets you write your website in multiple languages and switch between them with a single function call. All text content lives in JSON files — keeping it completely separate from your HTML and JS logic.

> **Used in Dream Chalet for:** Greek (primary) and English language support. Greek loads by default, the user's choice is saved in localStorage, and a GR|EN toggle in the nav switches the entire site instantly.

## File Structure

```
project/
├── locales/
│   ├── el.json   ← Greek (primary language)
│   └── en.json   ← English
└── js/
    └── i18n.js   ← i18next initialization
```

## Translation Files

```
// locales/el.json — Greek
{
  "nav": {
    "home": "Αρχική",
    "about": "Σχετικά",
    "gallery": "Γκαλερί",
    "amenities": "Παροχές",
    "contact": "Επικοινωνία"
  },
  "hero": {
    "title": "Dream Chalet",
    "subtitle": "Το τέλειο καταφύγιό σας στην καρδιά της Τρικαλινής φύσης",
    "cta_airbnb": "Κράτηση στο Airbnb",
    "cta_booking": "Κράτηση στο Booking"
  },
  "about": {
    "heading": "Ένα μέρος σαν όνειρο",
    "description": "Το Dream Chalet σας υποδέχεται σε ένα ζεστό, φιλόξενο περιβάλλον...",
    "bedrooms": "Υπνοδωμάτια",
    "bathrooms": "Μπάνια",
    "rating": "Βαθμολογία",
    "pets": "Κατοικίδια"
  },
  "booking": {
    "whatsapp": "WhatsApp",
    "viber": "Viber",
    "heading": "Κάντε Κράτηση",
    "subtitle": "Επικοινωνήστε μαζί μας για να κλείσετε τις ημερομηνίες σας"
  }
}
```

```
// locales/en.json — English
{
  "nav": {
    "home": "Home",
    "about": "About",
    "gallery": "Gallery",
    "amenities": "Amenities",
    "contact": "Contact"
  },
  "hero": {
    "title": "Dream Chalet",
    "subtitle": "Your perfect escape in the heart of Trikala",
    "cta_airbnb": "Book on Airbnb",
    "cta_booking": "Book on Booking"
  },
  "about": {
    "heading": "A Dream Come True",
    "description": "Dream Chalet welcomes you into a warm, inviting atmosphere...",
    "bedrooms": "Bedrooms",
    "bathrooms": "Bathrooms",
    "rating": "Rating",
    "pets": "Pets"
  },
  "booking": {
    "whatsapp": "WhatsApp",
    "viber": "Viber",
    "heading": "Make a Reservation",
    "subtitle": "Contact us to book your dates"
```

```
  }
}
```

## HTML — data-i18n attributes

```
<!-- Add data-i18n="key.path" to every text element -->
<nav>
  <a href="#about" data-i18n="nav.about">Σχετικά</a>
  <a href="#gallery" data-i18n="nav.gallery">Γκαλερί</a>
</nav>

<h1 data-i18n="hero.title">Dream Chalet</h1>
<p data-i18n="hero.subtitle">Το τέλειο καταφύγιό σας...</p>
<a data-i18n="hero.cta_airbnb">Κράτηση στο Airbnb</a>

<!-- Language toggle button -->
<button id="lang-toggle">
  <span class="lang-el">ΕΛ</span>
  <span> | </span>
  <span class="lang-en">EN</span>
</button>
```

## i18next Initialization (js/i18n.js)

```
// CDN scripts needed (add before your js/i18n.js):
// <script src="https://cdn.jsdelivr.net/npm/i18next@23/dist/umd/i18next.min.js"></script>
// <script src="https://cdn.jsdelivr.net/npm/i18next-http-backend@2/i18nextHttpBackend.min.js"></script>

const savedLang = localStorage.getItem("lang") || "el"; // Greek default

i18next
  .use(i18nextHttpBackend)   // loads JSON files via HTTP
  .init({
    lng: savedLang,           // starting language
    fallbackLng: "el",        // fallback if key not found in current lang
    backend: {
      loadPath: "/locales/{{lng}}.json"  // path pattern for translation files
    }
  })
  .then(() => {
    applyTranslations(); // apply on first load
    updateToggleButton(savedLang);
  });

// Apply translations to all data-i18n elements
function applyTranslations() {
  document.querySelectorAll("[data-i18n]").forEach(el => {
    const key = el.dataset.i18n;
    el.textContent = i18next.t(key);
  });
  // Update the html lang attribute (important for accessibility & SEO)
  document.documentElement.lang = i18next.language;
}

// Toggle between Greek and English
```

```
document.getElementById("lang-toggle").addEventListener("click", () => {
  const newLang = i18next.language === "el" ? "en" : "el";
  i18next.changeLanguage(newLang).then(() => {
    localStorage.setItem("lang", newLang);
    applyTranslations();
    updateToggleButton(newLang);
  });
});

function updateToggleButton(lang) {
  document.querySelector(".lang-el").style.fontWeight = lang === "el" ? "700" : "400";
  document.querySelector(".lang-el").style.color    = lang === "el" ? "#c9a96e" : "#9d8f7e";
  document.querySelector(".lang-en").style.fontWeight = lang === "en" ? "700" : "400";
  document.querySelector(".lang-en").style.color    = lang === "en" ? "#c9a96e" : "#9d8f7e";
}
```

> **Important:** i18next-http-backend fetches JSON files via HTTP. This works on GitHub Pages but NOT when you open index.html directly as a file ( `file://` protocol). For local development, use a simple server: run `npx serve .` in your project folder.

---

CHAPTER 12

# ical.js — Availability Calendar

**ical.js** is a JavaScript library that parses **iCal (.ics)** files — the calendar format used by Airbnb, Booking.com, Google Calendar, and virtually every other calendar system. By fetching the iCal feed from Airbnb and Booking.com, we can show a live availability calendar on the Dream Chalet site that updates automatically whenever a new booking is made.

> **Used in Dream Chalet for:** A monthly availability calendar in the Pricing section. Green = available, grey = booked. Updates automatically from Airbnb and Booking.com. The owner never needs to manually update the website.

## What is iCal?

iCal is a standard file format ( `.ics` ) for calendar events. Each booking platform generates a URL that serves a live `.ics` file for a property. Here's what it looks like inside:

```
BEGIN:VCALENDAR
VERSION:2.0
BEGIN:VEVENT
DTSTART:20260410
DTEND:20260415
SUMMARY:Reserved - Not available
END:VEVENT
BEGIN:VEVENT
DTSTART:20260420
DTEND:20260422
SUMMARY:Airbnb (HMXXXXXXXX)
```

```
END:VEVENT
END:VCALENDAR
```

## Getting iCal URLs from Airbnb and Booking.com

```
Airbnb:
  1. Go to airbnb.com → Hosting → Calendar
  2. Click "Availability settings"
  3. Scroll to "Sync calendars"
  4. Click "Export calendar" → copy the .ics URL

Booking.com:
  1. Go to Extranet → Calendar
  2. Click "iCal" or "Sync" option
  3. Copy the export URL
```

## Fetching and Parsing iCal

```
// Load ical.js from CDN:
// <script src="https://cdn.jsdelivr.net/npm/ical.js@1.5.0/build/ical.min.js"></script>

// iCal URLs (replace with real URLs from client)
const AIRBNB_ICAL_URL  = "https://www.airbnb.com/calendar/ical/YOUR_ID.ics";
const BOOKING_ICAL_URL = "https://ical.booking.com/v1/export?t=YOUR_TOKEN";

// CORS note: fetching from a different domain requires a proxy on GitHub Pages
// Use a free CORS proxy for now:
const PROXY = "https://corsproxy.io/?";

async function fetchBookedDates() {
  const bookedDates = new Set(); // stores "YYYY-MM-DD" strings

  // Fetch both calendars in parallel
  const [airbnbRes, bookingRes] = await Promise.allSettled([
    fetch(PROXY + encodeURIComponent(AIRBNB_ICAL_URL)),
    fetch(PROXY + encodeURIComponent(BOOKING_ICAL_URL))
  ]);

  for (const result of [airbnbRes, bookingRes]) {
    if (result.status !== "fulfilled") continue;
    const text = await result.value.text();

    // Parse with ical.js
    const parsed = ICAL.parse(text);
    const comp   = new ICAL.Component(parsed);
    const events = comp.getAllSubcomponents("vevent");

    events.forEach(event => {
      const vevent = new ICAL.Event(event);
      const start  = vevent.startDate.toJSDate();
      const end    = vevent.endDate.toJSDate();

      // Add every date in the range to the set
      const current = new Date(start);
      while (current < end) {
```

```
        bookedDates.add(current.toISOString().split("T")[0]); // "YYYY-MM-DD"
        current.setDate(current.getDate() + 1);
      }
    });
  }

  return bookedDates;
}
```

## Rendering the Calendar

```
async function renderCalendar(year, month) {
  const bookedDates = await fetchBookedDates();
  const calendarEl  = document.querySelector(".availability-calendar");
  calendarEl.innerHTML = "";

  // Month header
  const monthNames = ["Ιανουάριος","Φεβρουάριος","Μάρτιος","Απρίλιος",
    "Μάιος","Ιούνιος","Ιούλιος","Αύγουστος","Σεπτέμβριος",
    "Οκτώβριος","Νοέμβριος","Δεκέμβριος"];
  const header = document.createElement("h3");
  header.textContent = `${monthNames[month]} ${year}`;
  calendarEl.appendChild(header);

  // Day grid
  const grid = document.createElement("div");
  grid.className = "calendar-grid";

  const firstDay  = new Date(year, month, 1).getDay();
  const daysInMonth = new Date(year, month + 1, 0).getDate();
  const today = new Date().toISOString().split("T")[0];

  // Empty cells for days before month starts
  for (let i = 0; i < firstDay; i++) {
    grid.appendChild(Object.assign(document.createElement("div"), { className: "day empty" }));
  }

  // Day cells
  for (let day = 1; day <= daysInMonth; day++) {
    const dateStr = `${year}-${String(month+1).padStart(2,"0")}-${String(day).padStart(2,"0")}`;
    const cell    = document.createElement("div");
    cell.textContent = day;

    if (dateStr < today)         cell.className = "day past";
    else if (bookedDates.has(dateStr)) cell.className = "day booked";
    else                         cell.className = "day available";

    grid.appendChild(cell);
  }

  calendarEl.appendChild(grid);
}

// Render current month and next month
const now = new Date();
```

```
renderCalendar(now.getFullYear(), now.getMonth());
renderCalendar(now.getFullYear(), now.getMonth() + 1);
```

---

CHAPTER 13

# Putting It All Together — Dream Chalet Patterns

This final chapter shows exactly how all the libraries connect in the Dream Chalet project. These are the specific patterns you will implement, with everything wired together.

## 1. Project Bootstrap (js/main.js)

This is the first file that runs. It registers all plugins, initializes Lenis, and connects everything.

```
// js/main.js — runs first, sets everything up

// Register all GSAP plugins
gsap.registerPlugin(ScrollTrigger, Flip, Draggable, SplitText);

// ── Lenis smooth scroll ──────────────────────────
const lenis = new Lenis({
  duration: 1.2,
  easing: (t) => Math.min(1, 1.001 - Math.pow(2, -10 * t)),
});
gsap.ticker.add((time) => lenis.raf(time * 1000));
lenis.on("scroll", ScrollTrigger.update);
gsap.ticker.lagSmoothing(0);

// ── Nav anchor links use Lenis ──────────────────
document.querySelectorAll("nav a[href^='#']").forEach(link => {
  link.addEventListener("click", (e) => {
    e.preventDefault();
    lenis.scrollTo(document.querySelector(link.getAttribute("href")), { offset: -80 });
  });
});

// ── Init all modules ─────────────────────────────
initAnimations();   // js/animations.js
initScrollVideo();  // js/scroll-video.js
initThreeJS();      // js/three-bg.js
initMap();          // js/map.js
initCalendar();     // js/calendar.js
initI18n();         // js/i18n.js
initGallery();      // js/gallery.js
```

## 2. Transparent Nav → Solid on Scroll

```
// In js/animations.js
function initNav() {
```

```
  const nav = document.querySelector(".nav");

  ScrollTrigger.create({
    start: "top -80",
    end: 99999,
    onUpdate: (self) => {
      nav.classList.toggle("scrolled", self.progress > 0);
    }
  });

  // Mobile hamburger menu
  const hamburger = document.querySelector(".hamburger");
  const mobileMenu = document.querySelector(".mobile-menu");
  const menuTl = gsap.timeline({ paused: true });

  menuTl
    .to(mobileMenu, { x: "0%", duration: 0.5, ease: "power3.out" })
    .from(".mobile-menu a", { x: -30, opacity: 0, stagger: 0.07 }, "-=0.3");

  hamburger.addEventListener("click", () => {
    if (mobileMenu.classList.contains("open")) {
      menuTl.reverse();
      mobileMenu.classList.remove("open");
    } else {
      menuTl.play();
      mobileMenu.classList.add("open");
    }
  });
}
```

## 3. Hero Title Letter-by-Letter Reveal

```
function initHero() {
  const split = new SplitText(".hero-title", { type: "chars" });

  const tl = gsap.timeline({ delay: 0.3 });
  tl.from(split.chars, {
    opacity: 0,
    y: 80,
    rotationX: -90,
    transformOrigin: "0% 50% -50px",
    duration: 1,
    stagger: 0.03,
    ease: "power4.out"
  })
  .from(".hero-subtitle", { opacity: 0, y: 30, duration: 0.8 }, "-=0.4")
  .from(".hero-cta", { opacity: 0, y: 20, stagger: 0.15, duration: 0.6 }, "-=0.4")
  .then(() => split.revert()); // restore HTML after animation
}
```

## 4. Scroll-Scrubbed Video (js/scroll-video.js)

```
function initScrollVideo() {
  const video = document.querySelector(".scroll-video");
```

```
  // Video must be loaded before we can scrub it
  video.addEventListener("loadedmetadata", () => {
    gsap.to(video, {
      currentTime: video.duration,
      ease: "none",
      scrollTrigger: {
        trigger: ".scroll-video-section",
        start: "top top",
        end: "bottom bottom",
        scrub: 1,
        pin: true,
        anticipatePin: 1,
        onUpdate: (self) => {
          // Fade in/out text overlays based on scroll progress
          const progress = self.progress;
          gsap.to(".scroll-overlay-1", { opacity: progress < 0.3 ? progress / 0.3 : 1 - (progress - 0.3) / 0.2 });
          gsap.to(".scroll-overlay-2", { opacity: progress > 0.5 && progress < 0.8 ? (progress - 0.5) / 0.3 : 0 });
        }
      }
    });
  });

  video.load(); // ensure video starts loading
}
```

## 5. Premium Gallery with Flip (js/gallery.js)

```
function initGallery() {
  const strip      = document.querySelector(".gallery-strip");
  const overlay    = document.querySelector(".gallery-overlay");
  const closeBtn   = document.querySelector(".gallery-close");
  let   activeImg  = null;

  // ── Draggable horizontal strip ──────────────────────
  Draggable.create(strip, {
    type: "x",
    inertia: true,
    bounds: { minX: -(strip.scrollWidth - strip.parentElement.offsetWidth), maxX: 0 },
    edgeResistance: 0.85,
    cursor: "grab",
    activeCursor: "grabbing"
  });

  // ── Click image → Flip to fullscreen ────────────────
  strip.querySelectorAll("img").forEach(img => {
    img.addEventListener("click", () => {
      activeImg = img;
      const state = Flip.getState(img);

      overlay.appendChild(img);
      overlay.classList.add("active");
      lenis.stop(); // pause scroll while gallery is open

      Flip.from(state, {
        duration: 0.7,
        ease: "power2.inOut",
        absolute: true,
        onComplete: () => img.classList.add("expanded")
```

```
      });
    });
  });

  // ── Close → Flip back to strip ──────────────────────
  closeBtn.addEventListener("click", () => {
    if (!activeImg) return;
    activeImg.classList.remove("expanded");
    const state = Flip.getState(activeImg);

    strip.appendChild(activeImg);
    overlay.classList.remove("active");
    lenis.start(); // resume scroll

    Flip.from(state, { duration: 0.6, ease: "power2.inOut", absolute: true });
    activeImg = null;
  });

  // Close on Escape key
  document.addEventListener("keydown", (e) => {
    if (e.key === "Escape" && activeImg) closeBtn.click();
  });
}
```

## 6. Floating Sticky Book Button

```
function initStickyBookBtn() {
  const btn   = document.querySelector(".sticky-book-btn");
  const panel = document.querySelector(".sticky-book-panel");
  let   panelOpen = false;

  // Show button after scrolling past hero
  ScrollTrigger.create({
    trigger: ".hero",
    start: "bottom 80%",
    onEnter: () => gsap.to(btn, { opacity: 1, y: 0, duration: 0.5, ease: "power2.out" }),
    onLeaveBack: () => gsap.to(btn, { opacity: 0, y: 20, duration: 0.3 })
  });

  // Toggle booking panel
  btn.addEventListener("click", () => {
    panelOpen = !panelOpen;
    if (panelOpen) {
      gsap.to(panel, { y: 0, opacity: 1, duration: 0.4, ease: "power3.out" });
      gsap.from(panel.querySelectorAll("a"), { y: 15, opacity: 0, stagger: 0.08, duration: 0.3 }, "-=0.2");
    } else {
      gsap.to(panel, { y: 20, opacity: 0, duration: 0.3 });
    }
  });
}
```

## 7. Cursor Gallery in About Section

```
function initCursorGallery() {
  const items  = document.querySelectorAll(".room-list-item");
```

```
  const cursor = document.querySelector(".cursor-image");

  // quickTo = pre-cached, ultra-fast setter (much better than gsap.to in mousemove)
  const xTo = gsap.quickTo(cursor, "x", { duration: 0.5, ease: "power3.out" });
  const yTo = gsap.quickTo(cursor, "y", { duration: 0.5, ease: "power3.out" });

  document.addEventListener("mousemove", (e) => {
    xTo(e.clientX);
    yTo(e.clientY);
  });

  items.forEach(item => {
    const imgSrc = item.dataset.image;

    item.addEventListener("mouseenter", () => {
      cursor.src = imgSrc;
      gsap.to(cursor, { opacity: 1, scale: 1, duration: 0.3 });
    });

    item.addEventListener("mouseleave", () => {
      gsap.to(cursor, { opacity: 0, scale: 0.8, duration: 0.2 });
    });
  });
}
```

## 8. prefers-reduced-motion (Accessibility)

```
// Check if user has requested reduced motion (accessibility setting)
const prefersReducedMotion = window.matchMedia("(prefers-reduced-motion: reduce)").matches;

if (prefersReducedMotion) {
  // Disable GSAP defaults — everything just appears, no movement
  gsap.defaults({ duration: 0 });
  gsap.set(".hero-title, .section-title, .card", { opacity: 1 });
  // Don't init scroll video, Three.js, or Lenis
} else {
  // Full experience
  initLenis();
  initThreeJS();
  initScrollVideo();
}
```

**Final checklist before going live:**

- Remove all `markers: true` from ScrollTrigger instances
- Replace all placeholder images with client photos
- Replace all placeholder videos with client videos
- Fill in actual price per night, min stay, check-in times
- Confirm exact GPS coordinates for the map pin
- Add real Airbnb and Booking.com iCal URLs
- Test on iPhone and Android (check Lenis on touch, Three.js performance)
- Test `prefers-reduced-motion` in browser accessibility settings

- Run Lighthouse audit (aim for >90 Performance)

---

CHAPTER 14

# SEO Mastery

SEO (Search Engine Optimization) is how you make Google find your site, understand it, and rank it high. For Dream Chalet, good SEO means that when someone searches *"vacation rental near Meteora"* or *"ενοικίαση σαλέ Τρίκαλα"*, the site appears at the top. These 16 rules will be baked in from day one.

> **The goal:** A technically perfect site that Google can crawl, understand, and trust — plus a compelling appearance in search results that makes people click.

## Rule 1 — Semantic HTML Structure

Use the correct HTML tags for their purpose: `<header>`, `<nav>`, `<main>`, `<section>`, `<footer>`. Avoid putting everything in `<div>`.

**Why:** Google's crawler reads your HTML like a book. Semantic tags say "this is navigation", "this is the main content". Divs say nothing — it's a book with no chapters.

```
<!-- BAD -->
<div class="top"><div class="links">...</div></div>
<div class="stuff"><div class="text">...</div></div>

<!-- GOOD -->
<header>
  <nav>...</nav>
</header>
<main>
  <section aria-labelledby="about-heading">
    <h2 id="about-heading">About Dream Chalet</h2>
  </section>
</main>
<footer>...</footer>
```

## Rule 2 — One <h1> Per Page, Proper Hierarchy

**Why:** Google uses headings as a table of contents. Multiple `<h1>` tags confuse it. Never skip levels (don't jump from `<h1>` to `<h4>`).

```
<h1>Dream Chalet Trikala — Vacation Rental</h1> <!-- Only ONE -->
  <h2>About the Property</h2>
  <h2>Photo Gallery</h2>
  <h2>Nearby Attractions</h2>
```

```
    <h3>Meteora</h3>
    <h3>Elati</h3>
    <h3>Pertouli</h3>
    <h3>Palaiokarya Waterfalls</h3>
  <h2>Pricing & Availability</h2>
```

## Rule 3 — Title Tag (Most Important SEO Element)

**Why:** This is the blue clickable link in Google results. It's the #1 factor for understanding page topic and is the first thing searchers see.

```
<title>Dream Chalet Trikala | Vacation Rental near Meteora, Greece</title>
<!-- Rules: under 60 chars · main keyword first · location included -->
```

## Rule 4 — Meta Description

**Why:** Appears under the title in Google results. Doesn't affect ranking directly but affects *click-through rate* — a compelling description makes people click yours over competitors.

```
<meta name="description" content="Dream Chalet Trikala — 3-bedroom vacation rental with fireplace, garden & free park
<!-- 150–160 characters · include keywords naturally · add a CTA -->
```

## Rule 5 — <html lang=""> Attribute

**Why:** Tells Google and screen readers what language the page is in. A Greek page without `lang="el"` may not appear in Greek search results.

```
<html lang="el"> <!-- Greek by default -->

<!-- JS switches it when user toggles language -->
document.documentElement.lang = i18next.language; // "el" or "en"
```

## Rule 6 — hreflang Tags for Multilingual SEO

**Why:** Without hreflang, Google may treat the Greek and English versions as duplicate content and penalize one of them.

```
<link rel="alternate" hreflang="el" href="https://yourdomain.com/">
<link rel="alternate" hreflang="en" href="https://yourdomain.com/?lang=en">
<link rel="alternate" hreflang="x-default" href="https://yourdomain.com/">
```

## Rule 7 — Open Graph & Twitter Cards

**Why:** Controls how the site looks when shared on Facebook, WhatsApp, Viber, Telegram. A beautiful preview card with a photo = dramatically more clicks.

```
<!-- Open Graph (Facebook, WhatsApp, Viber, Telegram) -->
<meta property="og:title" content="Dream Chalet Trikala">
<meta property="og:description" content="3-bedroom rental near Meteora. Rated 9.8/10.">
<meta property="og:image" content="https://yourdomain.com/assets/images/og-image.jpg">
<meta property="og:image:width" content="1200">
<meta property="og:image:height" content="630">
<meta property="og:url" content="https://yourdomain.com/">
<meta property="og:type" content="website">
<meta property="og:locale" content="el_GR">
<meta property="og:locale:alternate" content="en_US">

<!-- Twitter Card -->
<meta name="twitter:card" content="summary_large_image">
<meta name="twitter:title" content="Dream Chalet Trikala">
<meta name="twitter:image" content="https://yourdomain.com/assets/images/og-image.jpg">
<!-- OG image: 1200x630px — best exterior photo of the chalet -->
```

## Rule 8 — JSON-LD Structured Data (Schema.org)

**Why:** Tells Google this is a vacation rental with a specific address, price, rating, and rooms. Enables **rich results** — star ratings, price, and images shown directly in search results — which massively increases clicks.

```
<script type="application/ld+json">
{
  "@context": "https://schema.org",
  "@type": "LodgingBusiness",
  "name": "Dream Chalet Trikala",
  "description": "3-bedroom vacation rental near Meteora, Greece",
  "image": "https://yourdomain.com/assets/images/hero.jpg",
  "address": {
    "@type": "PostalAddress",
    "addressLocality": "Raxa, Trikala",
    "addressRegion": "Thessaly",
    "addressCountry": "GR"
  },
  "geo": {
    "@type": "GeoCoordinates",
    "latitude": "39.6106",
    "longitude": "21.7500"
  },
  "telephone": "+306976908838",
  "url": "https://yourdomain.com",
  "aggregateRating": {
    "@type": "AggregateRating",
    "ratingValue": "9.8",
    "bestRating": "10",
    "ratingCount": "50"
  },
  "amenityFeature": [
    {"@type": "LocationFeatureSpecification", "name": "WiFi", "value": true},
    {"@type": "LocationFeatureSpecification", "name": "Free Parking", "value": true},
    {"@type": "LocationFeatureSpecification", "name": "Fireplace", "value": true},
    {"@type": "LocationFeatureSpecification", "name": "Garden", "value": true}
  ],
  "numberOfRooms": 3,
  "petsAllowed": true
```

```
}
</script>
```

## Rule 9 — Image Optimization

**Why:** `alt` text is how Google "sees" images. Uncompressed images slow your site. Slow site = lower ranking. WebP format is ~30% smaller than JPEG at the same quality.

```
<!-- BAD -->
<img src="photo1.jpg">

<!-- GOOD -->
<img
  src="assets/images/living-room.webp"
  alt="Spacious living room with fireplace at Dream Chalet Trikala"
  width="800"
  height="600"
  loading="lazy"
>
<!-- Hero image: no lazy loading (it's visible immediately) -->
<img src="assets/images/hero.webp" alt="Dream Chalet exterior, Trikala Greece" width="1920" height="1080">
```

## Rule 10 — Core Web Vitals

Google measures 3 performance metrics and uses them for ranking:

| METRIC | MEASURES | TARGET |
|---|---|---|
| LCP — Largest Contentful Paint | How fast the biggest element loads | Under 2.5s |
| INP — Interaction to Next Paint | How fast page responds to clicks | Under 200ms |
| CLS — Cumulative Layout Shift | How much layout jumps while loading | Under 0.1 |

```
<!-- Preload hero image for fast LCP -->
<link rel="preload" as="image" href="assets/images/hero-poster.webp">

<!-- Preload critical fonts (eliminates invisible text flash) -->
<link rel="preload" as="font" href="fonts/cormorant.woff2" crossorigin>

<!-- Always set width + height on images (prevents CLS) -->
<img src="gallery-1.webp" width="800" height="600" loading="lazy" alt="...">
```

## Rule 11 — Canonical URL

**Why:** Your site could be reached via http, https, www, non-www, with trailing slash, with `?lang=en` ... Google could see all of these as duplicate content pages. Canonical says "this one is the real one."

```
<link rel="canonical" href="https://yourdomain.com/">
```

## Rule 12 — Sitemap.xml + Robots Meta

**Why:** Sitemap tells Google every URL on your site. Robots tells Google what it's allowed to index. Both help the crawler do its job efficiently.

```
<!-- sitemap.xml (in root folder) -->
<?xml version="1.0" encoding="UTF-8"?>
<urlset xmlns="http://www.sitemaps.org/schemas/sitemap/0.9">
  <url>
    <loc>https://yourdomain.com/</loc>
    <lastmod>2026-04-02</lastmod>
    <priority>1.0</priority>
  </url>
</urlset>
```

```
<!-- In <head> -->
<meta name="robots" content="index, follow">
<link rel="sitemap" type="application/xml" href="/sitemap.xml">
```

## Rule 13 — HTTPS (Free on GitHub Pages)

**Why:** Google explicitly penalizes non-HTTPS sites. Browsers show "Not Secure" warnings. GitHub Pages provides free HTTPS automatically — no action needed.

> **Action:** In your GitHub repo → Settings → Pages → ensure "Enforce HTTPS" is checked.

## Rule 14 — Mobile-First Responsive Design

**Why:** Google uses the *mobile version* of your site for ranking. A bad mobile experience lowers your ranking even for desktop searches.

```
<!-- Required viewport meta tag -->
<meta name="viewport" content="width=device-width, initial-scale=1.0">
```

```
/* Mobile-first CSS — design for small screens first */
.gallery-strip { flex-direction: column; }

/* Then enhance for larger screens */
@media (min-width: 768px) {
  .gallery-strip { flex-direction: row; }
}

/* Touch targets must be at least 44x44px */
.nav-link, .btn { min-height: 44px; min-width: 44px; }
```

## Rule 15 — Page Speed Optimization

**Why:** Google ranks faster sites higher. Every extra second of load time costs you ranking and visitors.

```
<!-- defer = load after HTML parsed, execute after HTML ready -->
<script src="js/three-bg.js" defer></script>

<!-- async = load in parallel, execute immediately when ready -->
<script src="js/analytics.js" async></script>

<!-- Critical scripts: no attribute, at end of body -->
<script src="js/main.js"></script>
```

## Rule 16 — Favicon & Apple Touch Icon

**Why:** Google shows favicons next to URLs in search results. A missing favicon looks unprofessional and slightly hurts click-through rate.

```
<link rel="icon" type="image/svg+xml" href="/favicon.svg">
<link rel="icon" type="image/png" href="/favicon-32x32.png" sizes="32x32">
<link rel="apple-touch-icon" href="/apple-touch-icon.png" sizes="180x180">
<meta name="theme-color" content="#0e0b08">
```

## Full SEO Checklist

| # | RULE | IMPACT |
|---|---|---|
| 1 | Semantic HTML | High |
| 2 | One <h1>, proper hierarchy | High |
| 3 | Title tag (<60 chars, keywords first) | Very High |
| 4 | Meta description (150–160 chars) | High |
| 5 | html lang attribute | Medium |
| 6 | hreflang for GR/EN | High |
| 7 | Open Graph + Twitter Cards | High (social) |
| 8 | JSON-LD Schema (rich results) | Very High |
| 9 | Image alt + WebP + lazy loading | High |
| 10 | Core Web Vitals (LCP, INP, CLS) | Very High |
| 11 | Canonical URL | Medium |
| 12 | sitemap.xml + robots meta | Medium |
| 13 | HTTPS (free on GitHub Pages) | Very High |
| 14 | Mobile-first responsive | Very High |
| 15 | Page speed (defer, async, preload) | Very High |
| 16 | Favicon + Apple touch icon | Low |

> **After going live — verify with these free tools:**
>
> - **Google Search Console** — submit sitemap, monitor indexing, see search queries

- **PageSpeed Insights** (pagespeed.web.dev) — Core Web Vitals score
- **Rich Results Test** (search.google.com/test/rich-results) — verify Schema.org
- **Open Graph Debugger** (developers.facebook.com/tools/debug) — preview social cards

---

CHAPTER 15

# Advanced Features — Making It World-Class

These are the features that separate a *good* property website from an *award-winning* one. A preloader, custom cursor, PWA support, micro-interactions, cookie consent, accessibility, and analytics. Together they make the site feel professional, complete, and legally solid.

## 1. Preloader / Intro Screen

A branded loading animation shown while the page loads. It hides the ugly half-loaded state, builds anticipation, and ensures all animations start at exactly the right moment.

```
<!-- Add as first child of <body> -->
<div id="preloader">
  <svg class="preloader-logo" viewBox="0 0 200 60">
    <text x="10" y="45" class="logo-text">Dream Chalet</text>
  </svg>
  <div class="preloader-bar"><div class="preloader-fill"></div></div>
</div>
```

```
#preloader {
  position: fixed; inset: 0; z-index: 9999;
  background: var(--bg);
  display: flex; flex-direction: column;
  align-items: center; justify-content: center; gap: 2rem;
}
.preloader-bar { width: 200px; height: 1px; background: var(--border); }
.preloader-fill { height: 100%; width: 0; background: var(--gold); transition: width 0.3s; }
/* SVG text stroke animation */
.logo-text {
  font-family: "Cormorant Garamond", serif; font-size: 32px; fill: none;
  stroke: var(--gold); stroke-width: 1;
  stroke-dasharray: 1000; stroke-dashoffset: 1000;
  animation: draw 1.8s ease forwards;
}
@keyframes draw { to { stroke-dashoffset: 0; } }
```

```
// js/preloader.js
function initPreloader() {
  const preloader = document.getElementById("preloader");
  const fill = preloader.querySelector(".preloader-fill");
```

```
  // Simulate progress as resources load
  let progress = 0;
  const interval = setInterval(() => {
    progress = Math.min(progress + Math.random() * 15, 90);
    fill.style.width = progress + "%";
  }, 200);

  // When everything is loaded
  window.addEventListener("load", () => {
    clearInterval(interval);
    fill.style.width = "100%";

    // Short pause then reveal the site
    setTimeout(() => {
      const tl = gsap.timeline({
        onComplete: () => preloader.remove()
      });
      tl.to(preloader, { opacity: 0, duration: 0.6, ease: "power2.in" })
        .from(".hero-title", { opacity: 0, y: 60, duration: 1, ease: "power3.out" }, "-=0.2");
    }, 400);
  });
}
```

## 2. Custom Cursor

Replace the browser cursor with a small gold circle that morphs based on what the user hovers over. The signature of Awwwards-level sites.

```
<!-- Add right after <body> -->
<div id="cursor">
  <div class="cursor-dot"></div>
  <div class="cursor-ring"></div>
  <span class="cursor-label"></span>
</div>
```

```
#cursor { pointer-events: none; position: fixed; z-index: 9998; }
.cursor-dot {
  width: 6px; height: 6px; border-radius: 50%;
  background: var(--gold); position: fixed;
  transform: translate(-50%, -50%);
}
.cursor-ring {
  width: 36px; height: 36px; border-radius: 50%;
  border: 1px solid var(--gold); position: fixed;
  transform: translate(-50%, -50%);
  transition: width 0.3s, height 0.3s, opacity 0.3s;
}
.cursor-label {
  position: fixed; transform: translate(-50%, 24px);
  font-size: 0.65rem; letter-spacing: 0.1em;
  color: var(--gold); opacity: 0; transition: opacity 0.2s;
}
/* Hide default cursor everywhere */
*, *::before, *::after { cursor: none !important; }
```

```
// js/cursor.js
function initCursor() {
  // Skip on touch devices
  if (window.matchMedia("(hover: none)").matches) return;

  const dot   = document.querySelector(".cursor-dot");
  const ring  = document.querySelector(".cursor-ring");
  const label = document.querySelector(".cursor-label");

  const dotX  = gsap.quickTo(dot,  "x", { duration: 0.15, ease: "power3.out" });
  const dotY  = gsap.quickTo(dot,  "y", { duration: 0.15, ease: "power3.out" });
  const ringX = gsap.quickTo(ring, "x", { duration: 0.5,  ease: "power3.out" });
  const ringY = gsap.quickTo(ring, "y", { duration: 0.5,  ease: "power3.out" });
  const lblX  = gsap.quickTo(label,"x", { duration: 0.5,  ease: "power3.out" });
  const lblY  = gsap.quickTo(label,"y", { duration: 0.5,  ease: "power3.out" });

  document.addEventListener("mousemove", (e) => {
    dotX(e.clientX); dotY(e.clientY);
    ringX(e.clientX); ringY(e.clientY);
    lblX(e.clientX); lblY(e.clientY);
  });

  // State: hoverable link/button
  document.querySelectorAll("a, button, [data-cursor]").forEach(el => {
    el.addEventListener("mouseenter", () => {
      gsap.to(ring, { scale: 1.8, opacity: 0.5, duration: 0.3 });
    });
    el.addEventListener("mouseleave", () => {
      gsap.to(ring, { scale: 1, opacity: 1, duration: 0.3 });
      label.style.opacity = "0";
    });
  });

  // State: gallery drag
  const strip = document.querySelector(".gallery-strip");
  if (strip) {
    strip.addEventListener("mouseenter", () => {
      label.textContent = "Drag";
      label.style.opacity = "1";
      gsap.to(ring, { scale: 2.5, duration: 0.3 });
    });
    strip.addEventListener("mouseleave", () => {
      label.style.opacity = "0";
      gsap.to(ring, { scale: 1, duration: 0.3 });
    });
  }

  // State: gallery image view
  document.querySelectorAll(".gallery-img").forEach(img => {
    img.addEventListener("mouseenter", () => {
      label.textContent = "View";
      label.style.opacity = "1";
    });
  });
}
```

## 3. Micro-interactions

Small animations that make everything feel alive. The difference between a site that feels premium and one that feels static.

```
/* Scroll progress bar — thin gold line at top of viewport */
#scroll-progress {
  position: fixed; top: 0; left: 0; z-index: 999;
  height: 2px; width: 0%; background: var(--gold);
  transform-origin: left; transition: none;
}

/* Button press effect */
.btn { transition: transform 0.15s, box-shadow 0.15s; }
.btn:hover  { transform: translateY(-2px); box-shadow: 0 8px 24px rgba(201,169,110,0.25); }
.btn:active { transform: translateY(0);    box-shadow: none; }

/* Image hover zoom */
.gallery-img { overflow: hidden; }
.gallery-img img { transition: transform 0.6s cubic-bezier(0.25, 0.46, 0.45, 0.94); }
.gallery-img:hover img { transform: scale(1.04); }

/* Nav link underline reveal */
.nav-link {
  position: relative;
  &::after {
    content: ""; position: absolute; bottom: -2px; left: 0;
    width: 100%; height: 1px; background: var(--gold);
    transform: scaleX(0); transform-origin: right;
    transition: transform 0.3s ease;
  }
  &:hover::after { transform: scaleX(1); transform-origin: left; }
}
```

```
// Scroll progress bar
const progressBar = document.getElementById("scroll-progress");
window.addEventListener("scroll", () => {
  const scrolled = window.scrollY;
  const total    = document.body.scrollHeight - window.innerHeight;
  progressBar.style.width = (scrolled / total * 100) + "%";
}, { passive: true });

// Stat counters — animate numbers when scrolled into view
function animateCounter(el) {
  const target = parseFloat(el.dataset.target);
  const isDecimal = String(target).includes(".");
  gsap.to({ val: 0 }, {
    val: target, duration: 2, ease: "power2.out",
    onUpdate: function() {
      el.textContent = isDecimal
        ? this.targets()[0].val.toFixed(1)
        : Math.round(this.targets()[0].val);
    },
    scrollTrigger: { trigger: el, start: "top 80%", once: true }
  });
}
document.querySelectorAll("[data-target]").forEach(animateCounter);
```

# 4. PWA — Progressive Web App

Makes the website installable like an app and available offline — crucial for guests traveling in mountain areas with poor signal.

```
// manifest.json (in root folder)
{
  "name": "Dream Chalet Trikala",
  "short_name": "Dream Chalet",
  "description": "3-bedroom vacation rental near Meteora, Greece",
  "start_url": "/",
  "display": "standalone",
  "background_color": "#0e0b08",
  "theme_color": "#c9a96e",
  "orientation": "portrait",
  "icons": [
    { "src": "assets/icons/icon-192.png", "sizes": "192x192", "type": "image/png" },
    { "src": "assets/icons/icon-512.png", "sizes": "512x512", "type": "image/png" },
    { "src": "assets/icons/icon-512.png", "sizes": "512x512", "type": "image/png", "purpose": "maskable" }
  ]
}
```

```
<!-- In <head> -->
<link rel="manifest" href="/manifest.json">
<meta name="theme-color" content="#c9a96e">
<meta name="apple-mobile-web-app-capable" content="yes">
<meta name="apple-mobile-web-app-status-bar-style" content="black-translucent">
```

```
// sw.js — Service Worker (in root folder)
const CACHE = "dream-chalet-v1";
const OFFLINE_ASSETS = [
  "/", "/index.html", "/css/style.css",
  "/js/main.js", "/js/animations.js",
  "/assets/images/hero-poster.webp",
  "/locales/el.json", "/locales/en.json"
];

// Install: cache key assets
self.addEventListener("install", e => {
  e.waitUntil(
    caches.open(CACHE).then(cache => cache.addAll(OFFLINE_ASSETS))
  );
});

// Fetch: serve from cache, fall back to network
self.addEventListener("fetch", e => {
  e.respondWith(
    caches.match(e.request)
      .then(cached => cached || fetch(e.request))
      .catch(() => caches.match("/index.html")) // offline fallback
  );
});

// In main.js — register the service worker
if ("serviceWorker" in navigator) {
  window.addEventListener("load", () => {
```

```
    navigator.serviceWorker.register("/sw.js")
      .then(() => console.log("PWA: Service Worker registered"))
      .catch(err => console.error("SW registration failed:", err));
  });
}
```

## 5. Cookie Consent (GDPR — Required in Greece/EU)

EU law requires explicit consent before loading any tracking scripts. Greece is in the EU. This is not optional — without it the site is technically illegal for EU visitors.

```
<!-- vanilla-cookieconsent CDN -->
<link rel="stylesheet" href="https://cdn.jsdelivr.net/npm/vanilla-cookieconsent@3/dist/cookieconsent.css">
<script src="https://cdn.jsdelivr.net/npm/vanilla-cookieconsent@3/dist/cookieconsent.umd.js"></script>
```

```
// js/cookie-consent.js
CookieConsent.run({
  guiOptions: {
    consentModal: { layout: "bar", position: "bottom" }
  },
  categories: {
    necessary: { enabled: true, readOnly: true }, // always on
    analytics: { enabled: false }                 // opt-in
  },
  language: {
    default: "el",
    translations: {
      el: {
        consentModal: {
          title: "Χρήση Cookies",
          description: "Χρησιμοποιούμε cookies για να βελτιώσουμε την εμπειρία σας.",
          acceptAllBtn: "Αποδοχή Όλων",
          acceptNecessaryBtn: "Μόνο Απαραίτητα"
        }
      },
      en: {
        consentModal: {
          title: "Cookie Consent",
          description: "We use cookies to improve your experience.",
          acceptAllBtn: "Accept All",
          acceptNecessaryBtn: "Essential Only"
        }
      }
    }
  },
  onConsent: ({ cookie }) => {
    // Load GA4 only if user accepted analytics cookies
    if (CookieConsent.acceptedCategory("analytics")) {
      loadGoogleAnalytics();
    }
  }
});

function loadGoogleAnalytics() {
  const script = document.createElement("script");
  script.src = "https://www.googletagmanager.com/gtag/js?id=G-XXXXXXXXXX";
```

```
  script.async = true;
  document.head.appendChild(script);
  window.dataLayer = window.dataLayer || [];
  function gtag(){ dataLayer.push(arguments); }
  gtag("js", new Date());
  gtag("config", "G-XXXXXXXXXX");
  window.gtag = gtag;
}
```

## 6. Accessibility (a11y)

15% of people have some disability. Accessible sites rank better in Google, avoid legal risk, and show true craft. Inaccessible code is lazy code.

```
<!-- 1. Skip link — first element in body -->
<a href="#main-content" class="skip-link">Skip to content</a>

<!-- 2. ARIA labels on icon-only buttons -->
<button class="hamburger" aria-label="Open navigation menu" aria-expanded="false">
  <span></span><span></span><span></span>
</button>

<!-- 3. Gallery overlay — dialog role -->
<div class="gallery-overlay" role="dialog" aria-modal="true" aria-label="Photo gallery">
  <button class="gallery-close" aria-label="Close gallery"></button>
</div>

<!-- 4. Main content landmark -->
<main id="main-content">...</main>
```

```
/* Skip link — visible only when focused by keyboard */
.skip-link {
  position: absolute; top: -100%; left: 1rem;
  background: var(--gold); color: var(--bg);
  padding: 0.5rem 1rem; border-radius: 4px;
  font-weight: 600; z-index: 9999;
  transition: top 0.2s;
}
.skip-link:focus { top: 1rem; } /* appears when Tab is pressed */

/* Always show focus outlines for keyboard users */
:focus-visible {
  outline: 2px solid var(--gold);
  outline-offset: 3px;
  border-radius: 3px;
}
```

```
// Focus trap — keeps keyboard focus inside modal when open
function trapFocus(modal) {
  const focusable = modal.querySelectorAll(
    "a, button, input, [tabindex]:not([tabindex='-1'])"
  );
  const first = focusable[0];
  const last  = focusable[focusable.length - 1];
```

```
  modal.addEventListener("keydown", (e) => {
    if (e.key !== "Tab") return;
    if (e.shiftKey) {
      if (document.activeElement === first) { e.preventDefault(); last.focus(); }
    } else {
      if (document.activeElement === last)  { e.preventDefault(); first.focus(); }
    }
  });
  first.focus(); // set initial focus
}

// Update hamburger aria-expanded on toggle
hamburger.addEventListener("click", () => {
  const isOpen = mobileMenu.classList.toggle("open");
  hamburger.setAttribute("aria-expanded", isOpen);
});
```

## 7. Google Analytics 4 — Event Tracking

GA4 tracks who visits, where they come from, and what they do. The client can see how many people click "Book on Airbnb" vs WhatsApp, which photos they view most, and where they leave. This is how you prove the site works.

```
// js/analytics.js — called only after cookie consent
function initTracking() {
  if (typeof gtag === "undefined") return;

  // Track all booking button clicks
  document.querySelectorAll("[data-track]").forEach(el => {
    el.addEventListener("click", () => {
      gtag("event", "booking_click", {
        platform: el.dataset.track,   // "airbnb" | "booking" | "whatsapp" | "viber"
        location: el.dataset.location // "nav" | "contact" | "sticky"
      });
    });
  });

  // Track gallery opens
  document.querySelectorAll(".gallery-img").forEach((img, i) => {
    img.addEventListener("click", () => {
      gtag("event", "gallery_open", { image_index: i });
    });
  });

  // Track language switches
  document.getElementById("lang-toggle").addEventListener("click", () => {
    gtag("event", "language_switch", {
      from: i18next.language,
      to: i18next.language === "el" ? "en" : "el"
    });
  });

  // Track scroll depth (25%, 50%, 75%, 100%)
  const milestones = [25, 50, 75, 100];
  const reached = new Set();
  window.addEventListener("scroll", () => {
    const pct = Math.round(
      (window.scrollY / (document.body.scrollHeight - window.innerHeight)) * 100
    );
```

```
    milestones.forEach(m => {
      if (pct >= m && !reached.has(m)) {
        reached.add(m);
        gtag("event", "scroll_depth", { percent: m });
      }
    });
  }, { passive: true });
}
```

```
<!-- Add data-track attribute to all booking buttons in HTML -->
<a href="https://wa.me/306976908838" data-track="whatsapp" data-location="contact">WhatsApp</a>
<a href="https://www.airbnb.gr/..." data-track="airbnb" data-location="contact">Airbnb</a>
```

## 8. Performance Budget

Hard limits set from day one. Without a budget, it is easy to add one more library, one more animation, one more image — until the site takes 8 seconds to load and you have lost both the visitor and the ranking.

| ASSET | LIMIT | HOW TO ACHIEVE |
|---|---|---|
| Total page weight | < 3 MB | WebP images, compressed video |
| Hero video | < 5 MB | H.264, 720p, 10-20 sec loop |
| Each gallery image | < 200 KB | WebP, max 1200px wide |
| JavaScript total | < 400 KB | CDN, tree-shaking, defer |
| Lighthouse Performance | ≥ 90 | All rules in Ch.14 |
| Time to Interactive | < 3 sec | Preloader + preload + defer |

```
# Check your Lighthouse score before going live
# In Chrome DevTools → Lighthouse → Generate report
# Or use the CLI:
npx lighthouse https://yourdomain.com --only-categories=performance,accessibility,seo
```

**New files added to the project:**

- `manifest.json` — PWA app manifest
- `sw.js` — Service Worker for offline support
- `sitemap.xml` — SEO sitemap
- `js/preloader.js` — Intro screen
- `js/cursor.js` — Custom cursor
- `js/cookie-consent.js` — GDPR banner
- `js/analytics.js` — GA4 event tracking